

中图分类号: K909

论文编号: _____

学科分类号: 4133010

密 级: 公 开

安徽理工大学

专业学位硕士学位论文

(全日制)

基于 WebGL 的全球风场动态可视化方法研究

作者姓名: 张永强

专业名称: 测绘工程

研究方向: 地理信息技术与应用

导师姓名: 王波 教授

导师单位: 南京信息工程大学

答辩委员会主席: 董斌

论文答辩日期: 2022 年 12 月 7 日

安徽理工大学研究生院

2022 年 12 月 7 日

A Dissertation in surveying and mapping engineering

Research on global wind farm dynamic visualization
method based on WebGL

Candidate: Zhang Yongqiang

Supervisor: Wang Bo

School of Geomatics, Anhui University of Science and Technology,

Huainan, 232001, China

摘要

随着大数据、AI 人工智能等互联网技术的发展，数字地球开始多元化、信息化、智能化发展，数字地球也将更加侧重于平台化构建以及对社会福利等重大问题的解决，基于 B/S 架构的新一代数字地球产品正逐渐成为主要研究热点。但此架构存在两个主要弊端，第一个弊端就是 Web 端的动态可视化效果差，即科学计算方法对矢量场数据的渲染效果差，不能符合实际要求；第二个弊端就是大场景流畅渲染差，现如今比较常见的解决方法就是选取小场景进行动态可视化。因此本文针对 Web 端的动态可视化效果和数据渲染效率两个弊端，对风场数据采取数值积分法基于粒子系统进行可视化模拟并结合多种加速方法进行渲染加速，并结合以上实验方法和步骤实现一个基于 Web 端的全球风场数据动态可视化原型系统，本文所做的工作和得到的结论如下：

针对风场数据的渲染效果工作，利用双线性插值算法改善数据的位置分布情况，通过基于 WebGL 的龙格库塔算法对风场数据的位置点进行计算操作，对于轨迹生成效果差的情况通过改变积分步长的方式对算法进行改进，进而改善轨迹展示效果。经过实验得证，利用这种改进算法对粒子系统进行轨迹的模拟，是比较符合实际情况，可视化展示的效果比较细腻，可以得到满意的效果，因此本文所采用的可视化算法具有一定的创新。

针对 B/S 架构的计算效率弊端，本文通过 WebWorker 多线程方式提高计算效率，充分利用计算机的多线程并行计算能力，处理逻辑计算部分。针对 GPU 的图形加速能力，本文创建多个缓存区对象，在 GPU 中处理相关的粒子渲染工作，以此来减少 CPU 和 GPU 之间的数据交换工作，提高整个系统的计算效率。

总体来说，本文改进的四阶龙格库塔算法有效改进了可视化效果，多种加速算法有效提高了运行的流畅性，有效的解决 Web 端可视化中的两个主要问题，基于 Cesium 设计的原型系统具有良好的展示效果，对比 NullSchool 得知本文的原型系统具有良好的可视化展示效果，并结合全球地形要素符合地理信息技术的发展方向，符合数字地球的发展方向，具有一定的前瞻性，并且对于天气预报、气象分析等信息决策具有一定的参考意义，且对数字地球的建设具有重要意义。

图【43】表【3】参【8】

关键词：数字地球；矢量场；粒子系统；WebWorker；多帧缓存区；Cesium
分类号：K909

Abstract

With the development of big data, AI artificial intelligence and other Internet technologies, the digital earth has begun to diversify, informatize and intelligentize, the digital earth will also focus more on platform construction and the solution of major problems such as social welfare, and a new generation of digital earth products based on B/S architecture is gradually becoming the main research hotspot. However, there are two main drawbacks of this architecture, the first drawback is the poor dynamic visualization effect of the Web side, that is, the rendering effect of the scientific calculation method on the vector field data is poor, which cannot meet the actual requirements; The second disadvantage is the poor smooth rendering of large scenes, and the more common solution nowadays is to select small scenes for dynamic visualization. Therefore, aiming at the two drawbacks of dynamic visualization effect and data rendering efficiency on the web side, this paper adopts numerical integrals for visual simulation based on particle system and combines a variety of acceleration methods for rendering acceleration of wind field data, and combines the above experimental methods and steps to realize a global wind field data dynamic visualization prototype system based on the web side.

According to the rendering effect of wind field data, the bilinear interpolation algorithm is used to improve the position distribution of data, the position point of wind field data is calculated by the Longekuta algorithm based on WebGL, and the algorithm is improved by changing the integration step size for poor trajectory generation effect, so as to improve the trajectory display effect. After experimental evidence, the use of this improved algorithm to simulate the trajectory of the particle system is more in line with the actual situation, the visual display effect is more delicate, and satisfactory results can be obtained, so the visualization algorithm used in this paper has certain innovation.

In view of the disadvantages of computing efficiency of B/S architecture, this paper improves computing efficiency through WebWorker multi-threading, makes full use of the multi-threaded parallel computing power of the computer, and processes the logical computing part. In view of the graphics acceleration capability of GPU, this paper creates multiple buffer objects to process related particle rendering work in the GPU, so as to reduce the data exchange between the CPU and the GPU and improve the computing efficiency of the entire system.

In general, the improved four-order Longekuta algorithm in this paper effectively improves the visualization effect, a variety of acceleration algorithms effectively improves the smoothness of operation, effectively solves the two main problems in Web-side visualization, the prototype system designed based on Cesium has a good display effect, compared with NullSchool, we know that the prototype system in this paper has a good visual display effect, and combines global terrain elements to conform to the development direction of geographic information technology and the development direction of digital earth. It is forward-looking, and has certain reference significance for weather forecasting, meteorological analysis and other information decision-making, and is of great significance to the construction of digital earth.

Figure [43] Table [3] Reference [8]

Keywords: digital earth,vector field,particle systems,WebWorker, multiframe buffers, cesium

Chinese Books Catalog:_K909

目 录

1 绪论.....	1
1.1 选题背景.....	1
1.2 选题意义.....	1
1.3 国内外研究现状.....	2
1.3.1 三维矢量场动态可视化研究现状.....	2
1.3.2 龙格库塔算法研究现状.....	5
1.3.3 基于 B/S 架构的矢量场可视化平台	5
1.4 论文研究内容.....	8
1.4.1 矢量场数据及风场数据.....	8
1.4.2 风场数据动态可视化方法.....	8
1.4.3 粒子系统渲染加速算法.....	8
1.4.4 可视化原型系统构建.....	9
1.5 章节安排.....	10
2 矢量场数据及其风场数据概述	11
2.1 矢量场数据概述.....	11
2.2 风场数据概述及预处理.....	11
2.2.1 NetCDF 概述	11
2.2.2 风场数据预处理.....	13
2.3 本章小结.....	16
3 基于粒子系统的可视化方法研究	17
3.1 矢量场数据动态可视化的主要方法	17
3.1.1 点图标法.....	17
3.1.2 矢量线法.....	17
3.1.3 特征法.....	17
3.1.4 纹理映射方法.....	18
3.1.5 粒子系统法.....	18
3.2 基于粒子系统的风场数据动态可视化算法改进	19
3.2.1 双线性插值算法原理.....	20
3.2.2 四阶龙格-库塔算法原理	23
3.2.3 基于 WebGL 的变步长的龙格—库塔算法.....	24
3.2.4 基于颜色映射的风场数据可视化.....	26
3.3 实验结果和分析.....	27
3.3.1 原始算法可视化效果.....	27
3.3.2 算法改进可视化效果.....	29

3.3.3 颜色映射效果.....	29
3.4 本章小结.....	30
4 基于全球风场可视化场景的渲染加速方法	31
4.1 WebWorker 多线程.....	31
4.1.1 实验结果和分析.....	33
4.2 基于多帧缓存区的加速算法	34
4.2.1 实验结果和分析.....	35
4.3 本章小结.....	36
5 全球风场可视化原型系统设计与实现	37
5.1 基于 Cesium 的 B/S 架构系统构建方法	37
5.1.1 shader 自定义渲染	37
5.1.2 自定义 Primitives 的复杂几何体渲染	39
5.2 基于 Cesium 的风场可视化原型系统实现	40
5.2.1 基于粒子系统的风场数据可视化原型系统构建	40
5.2.2 总体设计.....	41
5.2.3 可视化系统模块实现.....	46
5.2.4 可视化系统结果与测试.....	50
5.3 系统对比分析.....	51
5.4 本章小结.....	54
6 结论与展望.....	55
6.1 结论.....	55
6.2 展望.....	56
参考文献.....	57
附录.....	63

插图清单

图 1 cesium 效果图	6
图 2 NullSchool 官网.....	7
图 3 Wind Map 官网.....	7
图 4 天地图 ^[64]	8
图 5 论文总体研究技术路线.....	9
图 6 某个区域随时间和高度变化的数组.....	12
图 7 CDL 文件结构	13
图 8 多种 NetCDF 数据类型	14
图 9 初始原始风场数据(U 方向)	14
图 10 解析数据集合	15
图 11 筛选数据集合	15
图 12 NetCDF 数据处理流程	16
图 13 点图标法.....	17
图 14 LIC 方法效果图	18
图 15 粒子系统流程图.....	19
图 16 线性插值原理.....	21
图 17 粒子更新流程图.....	22
图 18 颜色映射表.....	26
图 19 粒子颜色初始化.....	27
图 20 初步实现效果.....	28
图 21 采用固定步长的效果图.....	28
图 22 变步长的效果图.....	29
图 23 颜色映射的效果图.....	30
图 24 JavaScript 运行原理	31
图 25 单线程运行模式.....	32
图 26 多线程运行模式.....	32
图 27 风场轨迹生成过程(逻辑计算过程)	33
图 28 WebGL 的结构.....	34
图 29 多帧缓冲区算法.....	35
图 30 Cesium 体系架构.....	38
图 31 Cesium 渲染层结构.....	38

图 32 Primitives 结构图	39
表 3 风场数据表结构.....	41
图 33 原型系统架构图.....	42
图 34 系统技术结构图.....	44
图 35 风场数据处理流程.....	45
图 36 web 场景展示流程	46
图 37 初始化界面.....	47
图 38 初始化 GUI 界面.....	48
图 39 三维粒子风场数据可视化流程.....	49
图 40 三维粒子可视化展示.....	51
表 4 可视化系统对比分析结果.....	53
图 41 NullSchool 全球天气数据可视化.....	52
图 42 本文所设计的风场数据可视化.....	52
图 43 地形影像的风场可视化（多角度）	53

1 绪论

1.1 选题背景

在新技术不断更新、区域可持续发展、全球经济不断发展，发展形式变化莫测的背景下，国际数字地球学会的科学家团体提出一个基于数字地球的新概念——新一代数字地球^[1]。随着地球科学研究大气、陆地和海洋等相关问题，已经产生卫星观测、地面传感器和其它来源的地球数据，为地球科学系统性的深入研究带来新的机遇^[2]。新一代数字地球的发展需要现代信息技术和传统地理信息结合的发展，例如 Web2.0 和 Web3.0，实景三维、社会网络(Social Network)和 VGI(Volunteered Geographic Information)志愿地理信息等新地理信息理论和技术。

本文基于此背景选择全球风场可视化进而间接表达数字地球，通过全球风场表达数据之间的关系，通过相关技术使得自然学科和社会学科工作者共同联合发展，以此来克服全球对地观测分布式系统(GEOSS)^[3]在数据获取、描述、组织管理、地学服务、动态数据分析和建模、信息处理以及合理决策等方面的不足，使得全球风场可视化更具真实化、信息化，从而呈现我们一个更加智能化、虚拟现实化的信息数字地球。

1.2 选题意义

在地学领域中，随着数据的不断增长和实景三维的发展需求，矢量场可视化逐渐向三维立体化方向开始发展，被逐渐广泛用于智慧水利、天气气象、智慧城市等领域。通过渲染矢量场数据，描述这地理信息领域中非常常见的物理现象及一些相当重要的场景，尤其是在工程分析中发挥着重要的基础作用。

风场数据作为气象数据中重要的一部分，它的属性和结构比较复杂，无法直接观察数据间的联系和特点，因此就要选择某种可行的方法对数据进行渲染展示。随着互联网的发展，系统开始转向基于 B/S 架构进行设计，但因浏览器可视化效果差，加载大量数据效率差的问题，需要一种展示效果好，性能好，符合地理信息方向的矢量场可视化方法，来实现对矢量场数据的可视化效果。

因此针对这两个问题，本文提出对风场数据采用改进步长的四阶龙格库塔算法对可视化效果改进，采用 WebWorker 和多帧缓冲区加速算法进行加速渲染，以此来提高本文设计的原型系统的展示效果和流畅性。结合实验可知，本文提出的改进方法对矢量场的可视化具有一定的效果，并且采用的加速算法可以有效提高渲染效率，设计的原型系统具有良好的展示效果，并可以有效监测风场等相关物理现象的运动趋势，对数字地球的建设具有重要意义。

1.3 国内外研究现状

1.3.1 三维矢量场动态可视化研究现状

Laidlaw 研究比较 2D 矢量数据的六种可视化方法。这些方法可用于评估嵌入在 3D 中的 2D 表面上的 2D 矢量,以及为 3D 可视化方法定义类似任务^[4]。根据呈现效果,可将可视化分为静态和动态两种方式。根据表现形式加以区分,可以分为以下四大类^[5]:直接可视化、几何可视化、基于纹理的可视化和基于颜色、光学特征的可视化。这些方法优缺点明确,已在不同的应用领域得到广泛应用,取得不错的效果。

直接可视化主要包括点图标法。点图标法是实现最容易的、最直观的矢量场映射方法。该方法具有信息准确、直观等优点,但很难全面、连续地反应向量场,只适应于较简单的向量场可视化。黄晶慧将颜色映射与点图标映射方法相结合,最终实现旋风分离器速度场可视化^[6]。宋丽丽采用矢量箭头表示方向,颜色表示压强值,基于 OpenGL 对海洋向量场、等值面、彩色剖面进行可视化^[7]。郭晓非依据点图标映射法进行颜色映射,并用颜色的不同表示风速的大小,将处理后的每一时刻数据绘制成偏转箭头显示的形式,并基于 Cesium 平台上显示风场的走向,风速大小等信息^[8]。宋伟杰张量场的某一特征向量场在面图标上的方向信息,提出在面图标上通过绘制三维等值线及利用颜色属性来表现特征向量场的大小信息,这样,面图标就将特征向量场的方向和大小信息同时表现出来,从而进一步完善三维对称张量场的面图标方法,更全面地揭示张量场位于空间曲面上的信息^[9]。樊宇采用基于几何图形的点图标箭头法映射,并基于不同的颜色映射,结合几何着色器与 osgEarth 三维虚拟地球,提出实现一套海洋风场可视化的方法^[10]。王想红基于视点 LOD 的数据重采样,提出三维场景下改进三维箭头点图标法的矢量场动态可视化方法,以解决密集处的箭头重叠问题,以提升矢量场的可视化效果^[11]。

点图标法虽然绘制简单,并取得良好的效果,是基于数据简单,采样数据稀疏的情况下,因此会丢失向量场中的一些重要信息,流场中的一些某些特征也很难用点图标表法表达清除,不能保证全面地,连续的反应向量场特征。

几何可视化主要包括向量线、向量面法。点图标法无法直观的表示矢量场的连续性,为此有相关的科研人员针对矢量场可视化的流场可视化引入矢量线的表示方法。矢量线是对原始矢量场数据中抽取脉线、流线、迹线等几何体进行流场的可视化表达。Max 和 Crawfis 在 visualization 上提出在向量场中的面上来表示向量场中表面附近的向量方向可用头发作为图示,该方法在面上的采样点处画上一根根短线,然后让这些线沿向量的方向弯曲,和一般的头发比较类似^[12]。Yuan Chen 等人考虑矢量线生成算法的一些缺陷特征,采用一种新方法来自适应矢量线布局,用于 2D 和 3D 中的稳定矢量场。并同时考虑矢量线的欧式距离,形状和方向相似性的简单统计度量,提出一种定量误差度量来分配矢量线^[13]。Liya Li 根据在 3D 矢量场中播种矢量线会产生视觉上混乱的渲染结果,提出一种控制图像空间中矢量线的播种和生成的方法,以避免视觉混乱,可以直接实现矢量线的细节渲染、深度剥离和风格化渲染,从而更有效地可视化 3D 矢量场^[14]。Zockler 本文提出一种使用大量正确照明的流线进行交互式矢量场可视化的新技

术可以实现具有交互式帧速率的高渲染性能^[15]。

彭宝云针对矢量线可视化的数据输入/输出过程中的瓶颈问题,提出一种大规模矢量线计算的数据预取方法,在进行矢量线计算时可提前预取计算所需的数据块,可以有效地提高大规模流场矢量线可视化的效率^[16]。在速度变化矢量变化很快的地方,误差就会变得很大,因此 Kenwright^[17]在前人开发的数值生成矢量线的生成方法,提出基于流函数的矢量线构造方法,不需要按照步长方法进行积分,避免数值积分导致的误差,从而消除传统跟踪算法的计算密集部分。Feng^[18]等人提出一种基于非结构化三角形网格的底层流线追踪算法。该方法可以将单元格中矢量线的运动轨迹在不需网格单元面上的信息即可准确描述种子点的运动时间和离开单元格时的坐标,可极大地避免种子点追踪时产生的误差。Ji 等人^[19]提出一种基于四面体的不规则流场流线生成算法。该方法的种子点放置位置由空间分布因子决定,可展示不同流线细节,因此可避免流线提前终止产生的特征信息丢失等问题,可反映洋流特征。Yeh 提出一种新方法,该方法非常精确地控制每个矢量场中一系列流线的美学布局和艺术绘制,以模仿催眠线条艺术的风格^[20]。Ali 基于积分的几何流可视化探索流场结构,通过在每个计算节点的多核上并行化多个进程,实现最大性能^[21]。

葛成^[22]认为矢量线方法较点图标法虽有很大的改进,能表示矢量场的连续性,绘制效果也较后者好,通过初始点的位置适当选取,可以显示矢量场的特征和细节,但是不能很好的表示矢量场的量的大小和直观的显示矢量场的运动情况,不符合矢量场所代表的实际情况。综上所述,矢量线方法的绘制质量主要取决于种子选取的质量即初始点的位置选取,如果种子点选取不合适就会丢失矢量场的一些重要的场信息等。

基于颜色、光学特征的可视化方法是非常常见的一种矢量场映射方法。矢量场数据不同于标量场数据中的标量值和颜色之间的一一映射的关系即大小,还包括方向这个重要特征。矢量的大小、方向矢量的点积映射为颜色值。Sakas 使用体绘制方法成功地将湍流气体的运动动态地模拟出来,他用频谱合成技术生成随机的、随时间变化的密度场^[23]。田玲^[24]基于二维纹理映像的体绘制算法进行改进,提出变换纹理序列的思想对并且通过计算体数据包围盒与垂直与视线的平面的切片多程序设计实现三维纹理映像体绘制算法,实现较好的绘制效果。Max 和 Crawfis 通过体绘制纹理验证风速方向和风平流的方法对三维体纹理进行渲染,能较好地显示出来速度的方向^[25]。Peng 找出一种四叉树搜索方法,在空间位置和尺度的基础上,通过动态读取和显示,达到质量点云可视化的目的^[26]。动态体绘制技术可生成具有较高真实感效果的图形。但也存在问题,对场中的标量进行动态体绘制尝尝会使人产生错觉,并没有显示矢量却也可能产生具有方向的效果^[27]。

对于此类的矢量场可视化方法还有一个应用较多的方法就是粒子方法。1983 年,Reeves 提出粒子方法^[28],一种对不规则模糊物体的建模方法。即此后,粒子方法开始被引进矢量场的可视化领域。Reeves 又对扩展他提出的粒子理论,将粒子系统中的粒子由发光体推广到可反射、可折射外部光的实体^[29]。研究人员也根据他的粒子理论,将粒子系统应用到洋流,大气等较为复杂的可视化领域^[30]。Van Wijk 基于粒子理论提出具有能够反射有向光源和法线,并且面积非常小的面粒子,每个面粒子都有几何和时间属性^[31]。Chandler 研究基于插值的

替代方法,通过平滑粒子流体动力学 (SPH) 等模拟技术生成的可压缩和不可压缩粒子系统中的轨迹生成^[32]。

随着粒子理论的完善,粒子已能较好地模拟出模糊物体(如雨,雾,云等)的真实性。Sims 将每一个粒子都设置头和尾,并且都有各自的位置、颜色、半径和透明度等属性,可实现粒子的并行绘制,使得粒子的绘制更加的灵活^{[33]-[34]}。刘恒星提出一种将 SPH 的流场结果与粒子系统相结合,运用 CFD 对流场进行求解,在 Cesium 上实现风场的多角度可视化^[35]。Grottel 针对高性能图形进行低级 GPU 优化的可能性开发 MegaMol,用于基于粒子的数据的可视化研究,用于数据处理和高级呈现实现的通用功能对所有应用程序都可用且有益^[36]。方京针对基于粒子系统的动态流场可视化在粒子管理体系和制图表达效果上存在的问题,提出一种优化粒子管理和制图表达的方法,结果表明本文所提方法有效提高基于粒子系统的流场动态可视化的制图效果^[37]。韩吉焯针对水流的真实性和系统的性能问题,通过 SPH 和粒子系统对水流进行模拟,在 Unity3D 中实现水流墙可视化^[38]。

基于纹理的可视化方法,纹理映射在图像空间进行,因而具有图像空间的连续性,可生成具有图像空间分辨率的细致图形。主要包括点噪声法、线积分卷积法和可变形参数域卷积法。Van Wijk 基于强度函数产生的纹理,并基于此纹理反映流场,因此提出点噪声法^[39]。Khorloo 等人提出一种简单的方法来生成三维冻结和非冻结湍流风场,用于风致运动的动画,通过采用基于噪声的方法和考虑应用于结构工程领域的实际风的关键特性,能够在室外风场模拟中产生合理的结果^[40]。Leedom 与 Cabra 首次提出线积分卷积法^[41]。Dwivedi 实现针对任何位图像和任何分辨率设计的算法。纹理增强可视化效果,有助于区分图像部分^[42]。王海洋针对传统 Volume LIC 算法应用于三维流场时出现的遮挡问题,提出基于稀疏噪声 Volume LIC GPU 加速绘制技术^[43]。Liao 提出一种基于并行 3D 线积分卷积 (LIC) 的新型流动可视化方法。并通过实验得知 3D LIC 可视化被证明是一种高效且平衡良好的并行流可视化^[44]。Zhang 将多个一维纹理映射到流线,以加速计算速度,分析结果表明,ESTS 产生更好的视觉结果,并且 ESTS 足够快,可以交互式地可视化矢量场^[45]。Huang 利用执行标准图像空间 LIC 以生成流动纹理,通过利用视点依赖性向后跟踪和 mipmap 噪声相位,以实现非稳定流动的连贯可视化^[46]。Maskey 探讨纹理的方向性在多元可视化中的作用,描述并展示一种新的基于定向纹理的数据可视化技术^[47]。Kim 在透视投影的条件下,当表面使用包含沿第一和第二主方向的定向能量的图案进行纹理时,得知如果图案只包含沿一个主要方向的标记,或者除主要方向之外还包含沿其他方向的信息,则总体性能较差^[48]。Wang 考虑矢量场所承载的信息量和基于扩展信息熵的纹理,提出一种三维矢量场模糊特征可视化的纹理适应技术,从而基于模糊特征测量和信息量产生更合理的可视化结果^[49]。

基于纹理的可视化矢量场映射方法能比较好地表现出矢量场的效果,基于纹理的映射方法,克服集合映射和基于颜色特征方法两者的缺点,具有可观的应用场景,可使这种方法计算量复杂,计算难度大,需要开发人员要有较高的编程能力,尤其是对从事地理信息研究的相关人员的研究水平和编程水平要求特别高,同时需要利用 GPU 图形硬件加速和特定的加速算法,因此对计算机的硬件性能要求比较高,成本过高,不适合对地理信息对实际生活的应用场景。

1.3.2 龙格库塔算法研究现状

J. W. V. Dambros^[50]针对海洋生态系统漏油问题,通过 Runge-Kutta 求解微分方程,对 ECOS-简易耦合油数值模型进行修改,以改善模拟并使结果与真实场景相似。卢祥将龙格库塔法引入 GLONASS 轨道仿真,研究步长参数设置对结果的影响,结果表明龙格库塔法适用于 GLONASS 轨道仿真^[51]。Huo 采用 RK4 方法求解明流中稳态逐渐变化的水面剖面剖面计算模型,既适用于棱柱形通道,也适用于非棱柱形通道,并以 Runge-Kutta 方法作为算法内核,选择 MATLAB 中的 ode45 计算任何控制控制段的水面线,避免标准步骤方法产生的问题^[52]。Hongyu 针对非线性波动方程的模型问题,讨论方程的不同多辛重构,探索多辛 Runge-Kutta 方法,并构造一些流行且有效的多辛方案,并针对这些方案执行稳定性分析^[53]。Muhammad 针对粒子模拟的不同运行形式,采用不同的数值积分方法,特别是针对粒子的极端运动采用四阶龙格库塔算法进行仿真计算,并用 C++ 进行实现,结果表明此数值积分方法适用于粒子的极度突然运动^[54]。Yang Cao 将非线性近方法与 Runge-Kutta 方法结合,开发用于求解刚性微分方程的 A-稳定显示非线性 Runge-Kutta 方法,构建三阶公式,避免求解隐式方法必须求解的非线性方程^[55]。Abbas 提出一种新的 4 阶分段 Runge-Kutta 方法的推导,用来解决时间分段初始值问题,并与精确值对比,结果表明该方法的稳定性和对数值计算的效率和准确性^[56]。K. Uçak,对所引入的调整机构在不同情况下的非线性三罐系统性能进行检验,结果表明龙格-库塔模型具有良好的控制和识别性能^[57]。富明慧提出求解非线性动力学方程的一种改进的精细龙格库塔法,与已有的精细龙格库塔法相比,在精度和效率上均有较大程度的改善^[58]。Wang 本文提出一种高精度的龙格-库塔神经网络(RKNN)用于识别由常微分方程描述的未知动力系统。针对 RKNN 研究两种类型的学习算法,梯度和非线性递归最小二乘算法。学习算法的收敛性分析是理论上完成的。计算机模拟证明 RKNN 的已证明特性^[59]。

通过国内外关于龙格库塔算法的研究可知,此方法模拟的场景较为真实,是一种合适的计算机可视化方法。但相关的人员只是将四阶龙格库塔算法应用到可视化场景中,对四阶龙格库塔算法相结合其他方法改进,或者采用高阶龙哥库塔算法针对离散程度高的样本数据进行验证,虽然在精度上有较大的改进,但对于量大且速度高的数据容易造成视觉眩晕,特别是旋涡处或者其它复杂区域,因此关于现有的龙格库塔改进方法只适合数据量少、离散程度高的小数据样本,不能有效满足风场数据的数据量大且速度快的特点,因此本文采用更为精度较大的四阶龙格库塔算法可视化,并根据本文所用的风场数据的特点,对此方法的步长即截断误差进行改进,并将此方法应用到可视化场景中,进而改善风场复杂区域表现的效果。

1.3.3 基于 B/S 架构的矢量场可视化平台

由于 WebGL 的强大功能和 Chrome 浏览器强大内核渲染能力,这让 Web 端的三维渲染功能变得越来越强大和完善,也让人们感到越来越方便携带和安装,使得现在的研究人员开始转向 WebGIS 的三维方向开发,并取得一系列成功的产品。

1)Cesium

Cesium 是 AGI 公司计算机图形开发小组与 2011 年研发的跨平台、跨平台展示三维地球和地图可视化等多种 GIS 要素的开源 JavaScript 库(CesiumJS)。Cesium 支持 2 维、2.5 维、3 维形式的地图展示,可以绘制各种几何图形、高亮区域,并支持图片和三维模型和海量三维模型数据、影像数据、地形数据、矢量数据等丰富的地理数据的高效可视化展示^[60],在地形仿真、智慧城市、指挥交通等领域具有非常广泛的应用。Cesium 为三维 GIS 提供一个多种数据类型可视化平台。

Cesium 无需安装任何插件就能在新版浏览器上运行,并且各种功能特点非常适合 GIS 方向的人员进行使用,也非常适合地理空间数据在各种测绘图层上进行展示,是非常有前景的 WebGIS 三维展示库,是在开源 Apache2.0 许可下,可免费用于商业和非商业用途^[61],如图 1。

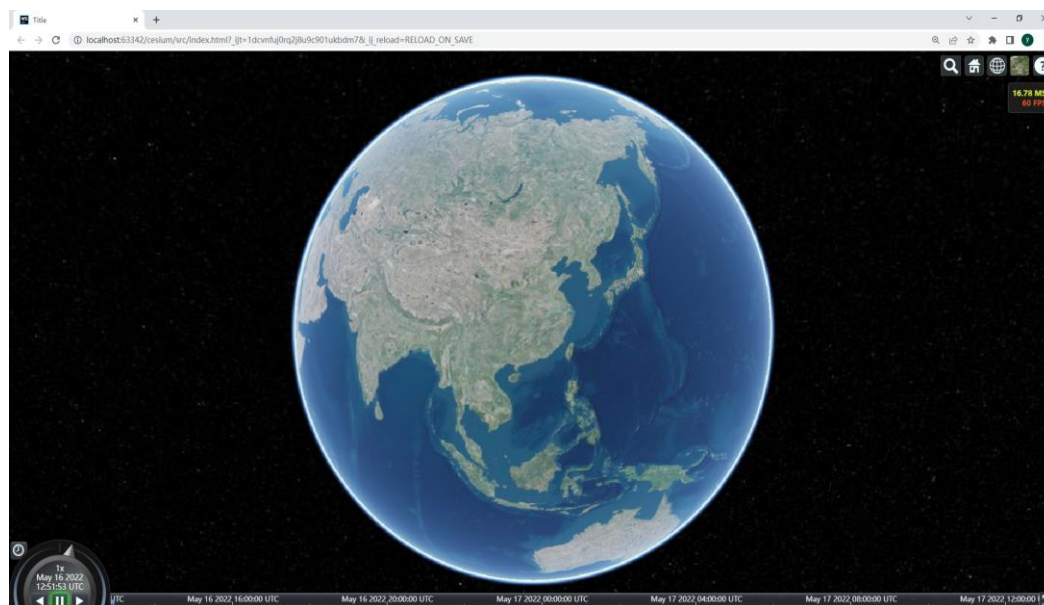


图 1 cesium 效果图

Figure 1 cesium renderings

2)NullSchool

NullSchool^[62]是一个全球天气状况的视觉化产品,采用超级计算机进行预报,每三个小时更新一次。对海洋平面每五天更新一次,海洋平面温度每天更新,对于海浪的变化每三个小时更新,极光每三十分钟更新一次,支持多种模式和多种数据源,但是只能单一角度的观看,并且没有地形的影响无法从三维 GIS 的角度出发进行考虑。如图 2。

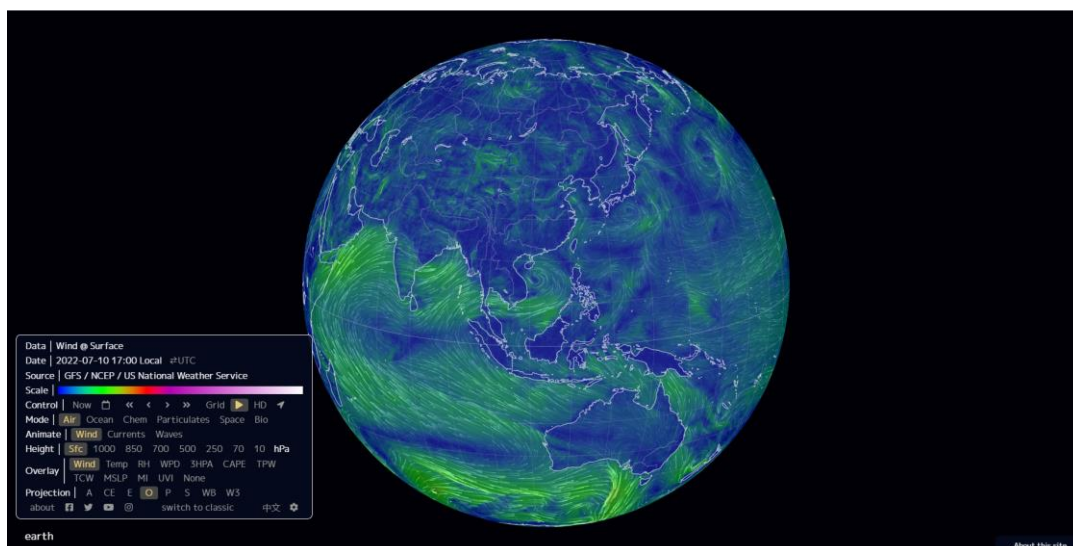
图 2 NullSchool 官网^[62]

Figure 2 Three.js official website

3) Wind Map

可根据用户所处的位置进行搜索当地的卫星图像和查看当地的天气状况，但是 Wind Map^[63]是商用的，非开源，用户想要更高级的高能必须定制，使用非常不方便，如图 3。

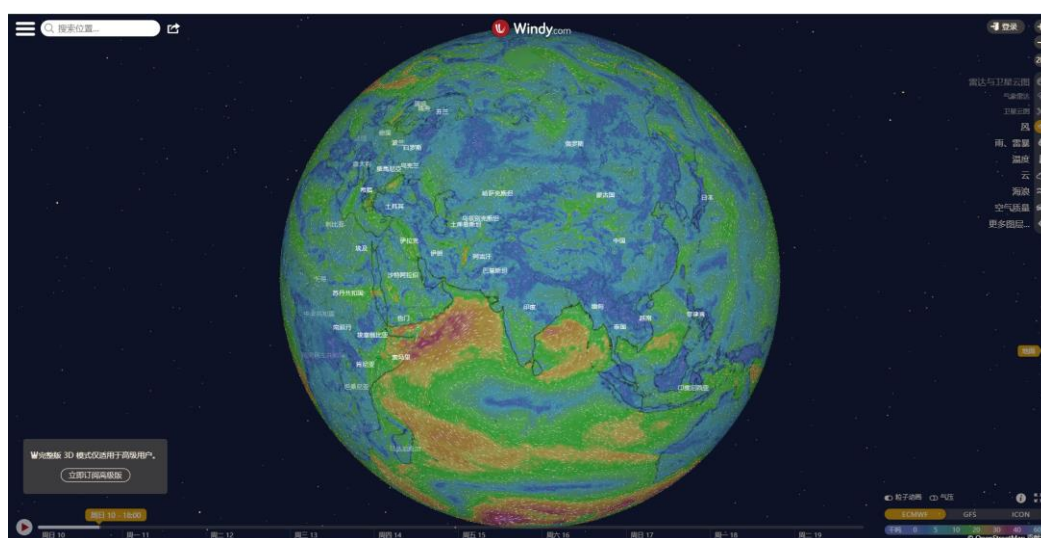
图 3 Wind Map 官网^[63]

Figure 3 Wind Map official website

4) 天地图

国内的天地图^[64]的三维地球，和其它基于 B/S 架构的三维地理信息可视化平台一样，有图层管理，地图整饰、空间量测等操作，此平台是采用瓦片层次的形式进行设计，运行比较流畅，但交互性上的功能扩展比较差，不能很好和其他平台进行对接和扩展，如图 4。

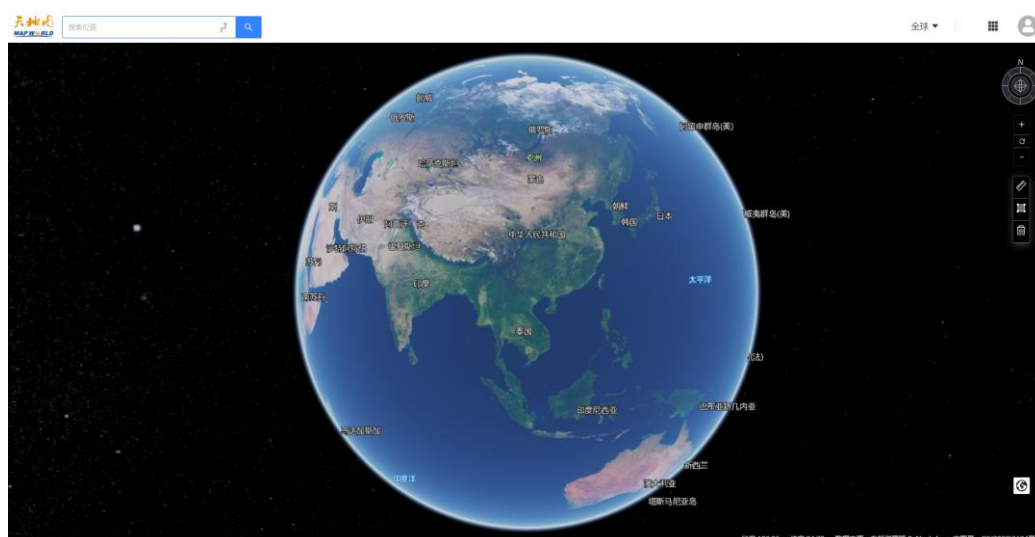
图 4 天地图^[64]

Figure 4 Tiandimap official website

1.4 论文研究内容

本文根据风场数据的动态可视化设计的原型系统思路主要分为四部分。

1.4.1 矢量场数据及风场数据

论述矢量场数据的特点和结构，体现出矢量场数据在三维可视化中的优越性，介绍风场数据的结构和属性特点，其次对本文所需要的风场数据从获取、筛选、格式转化和入库等一系列操作进行实现，删除一些不满足效果的数据点，然后通过插值算法对风场数据进行处理以此来提高数据的分辨率，以此来满足本文算法对数据的要求。

1.4.2 风场数据动态可视化方法

研究基于 WebGL 的四阶龙格库塔算法，根据风场数据渲染会出现偏移等情况，提出变步长的四阶龙格库塔算法，该改进的算法主要用于粒子初始化更新和渲染过程，用于缩进或延长步长，进而改善粒子轨迹效果。该改进算法是基于风场数据特点进行可视化表达渲染，是用于粒子更新过程中的产生的速度和位置变化过程的计算过程。本文对四阶龙格库塔算法的步长进行改变，并给出该改进算法的推导过程，由实验得知此方法的可视化展示效果好，并且此方法是基于 GPU 进行的多帧缓冲区数据存储交换方法，计算效率较高。

1.4.3 粒子系统渲染加速算法

由于计算机的硬件和软件等条件的缺陷，实时更新各个粒子状态变得困难，因此要结合相应的加速方法以得到渲染要求。基于此，本文结合 WebWorker 多线程技术和基于 GPU 的多帧缓冲区加速算法，这两种加速算法使得风场动态可视化原型系统能够流畅运行和交互，以此来满足全球范围内的风场数据模拟的真实性、运行的高效性和设计的合理性。

1.4.4 可视化原型系统构建

本文基于要体现出风场的运动趋势，构建 B/S 架构的全球风场原型系统，因此通过自定义 Primitives 的和 shader 的管道渲染方法，采用 ESMModule 模块化方式对代码进行重构，研究构建一个面向 GIS 领域的全球风场原型系统。此内容包括主要着色器、纹理映射等关于管线渲染的技术流程及其三维风场可视化流程，同时在 ESMModule 模块化代码组织规范下，对 Cesium 的底层架构进行自定义设置，进而实现构建本文所需要实现面向 GIS 领域的全球风场原型系统。

论文的总研究思路，如图 5。

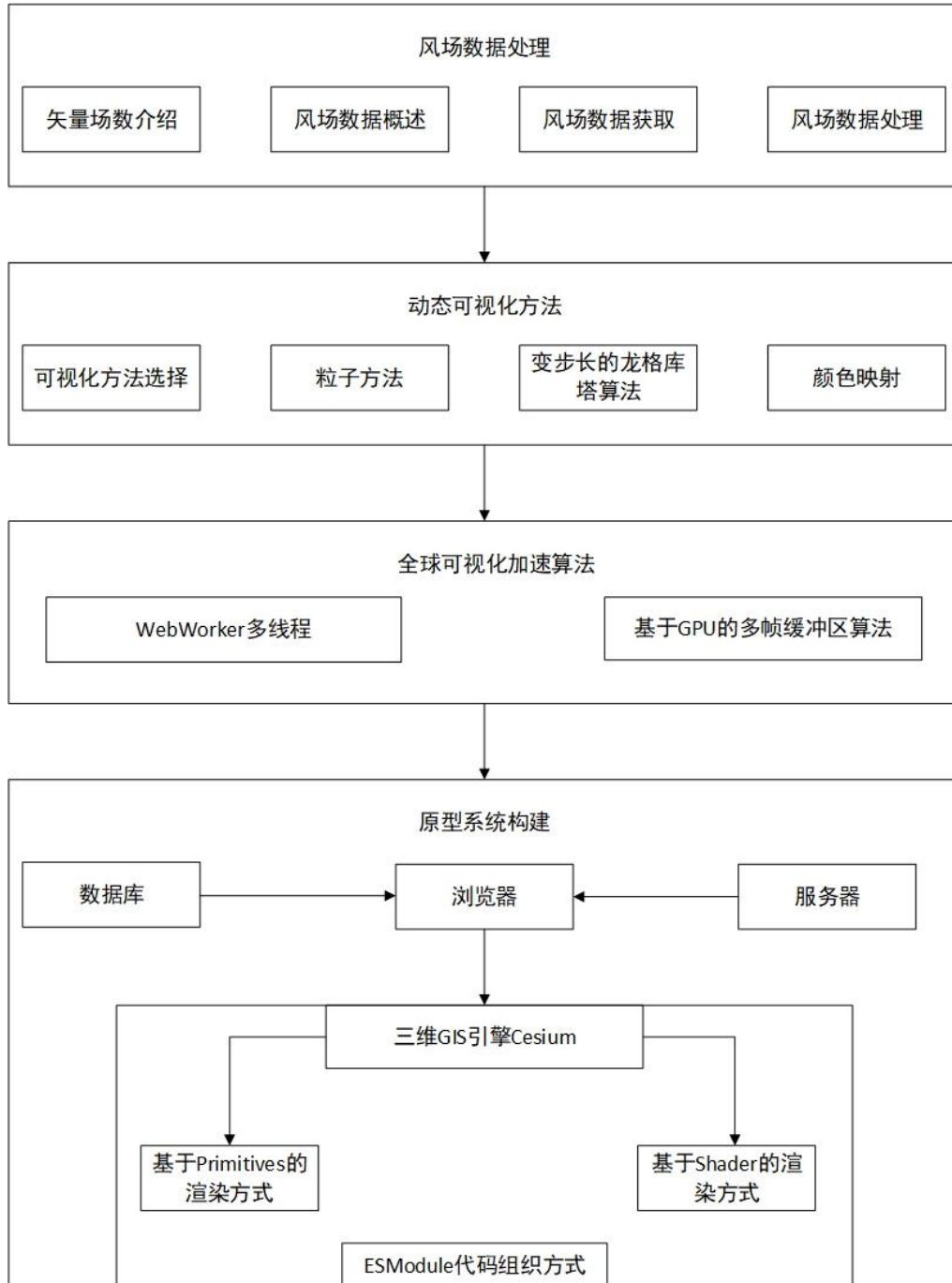


图 5 论文总体研究技术路线

Figure 5 The overall research idea framework of the paper

1.5 章节安排

根据本文的研究内容，本文通过六个章节进行描述，具体章节安排如下，

第一章，绪论。介绍本文的研究背景和研究意义以及相关的国内外矢量场动态可视化方法，并阐述关于四阶龙格库塔算法的发展现状以及关于 B/S 架构的 Web 数字地球平台。

第二章，风场数据概述及预处理。阐述矢量场数据以及相关的风场数据及其格式、存储特点，最后针对数据结构特点处理风场数据。

第三章，基于粒子系统的可视化方法研究。比较可视化方法，选择最为合适的粒子系统；介绍双线性插值算法和基于 GPU 的四阶龙格库塔算法，并针对数据的空间分布和属性特点，对四阶龙格库塔算法进行改进，改变其算法的步长，进而改变其粒子系统的表现效果。

第四章，全球风场渲染加速算法。主要采用两种渲染加速算法，其中第一种方法是 WebWorker 多线程，用于本文的位置逻辑计算部分；第二种加速方法是基于 GPU 的多帧缓冲加速算法，用于减少数据交换和降低粒子渲染过程中所产生的内存消耗，这两种方法的最终目的都是为保证和提高原型系统的整体运行效果。

第五章，风场可视化原型系统的设计与实现。设计基于三维渲染引擎 Cesium 构建 B/S 架构的风场可视化原型系统，本文基于 Cesium 源码进行重构设计和封装，基于 primitives 和 shader 进行多管道渲染，并采用 ESModule 的代码组织方式对本文的方法进行设计，对整体的场景进行实现，并展示实验结果。

第六章，结论与展望。对本文的工作进行总结，对所涉及的问题进行分析，给出相对应的解决方案，并针对本文所设计的原型系统进行更远一步的展望工作。

2 矢量场数据及其风场数据概述

三维矢量场数据的映射一直是三维矢量场可视化研究的热点所在^[66]。随着计算机的发展, 矢量场数据的可视化从刚开始的试验方法转向为由计算机实现可视化。

2.1 矢量场数据概述

矢量场数据类型不仅包括表示大小的标量数据, 还包括具有方向的矢量数据。这些数据点的分布具有不规则性, 导致原始的矢量场数据无法直接输入到可视化软件中加以分析处理, 因此必须对这些数据进行提前处理, 转化为有利于可视化软件分析处理的数据形式。

在科学计算中, 所研究对象的特性往往是用常微分方程、积分方程或者非线性方程来描述的。对于这些方程的离散数值解, 要用数值求解方法求解出函数值, 因此可视化的对象一般是指空间上的离散数据, 矢量场数据经过科学分析得出来的都是离散数据点。这种数据之间的连接关系类型主要包括以下几种^[67]:

(1) 结构化数据

结构化数据是指, 是矢量场可视化中最常用的数据类型, 在逻辑上组织成三位数组的空间离散数据^[68]。这些数据可通过空间离散数据的各个元素的列号、层号和行号进行搜索。又根据其中的各元素的不同物理分布分为结构规则网格化数据、结构均匀网格化数据、结构矩形网格化数据和不规则网格结构化数据。

(2) 非结构化数据

这种类型的空间数据是由一系列单元, 比如三棱柱、四棱锥、六面体或者四面体等构成的。这些单元的面可以是平面或者曲面, 具有灵活性但计算较为耗费内存。

(3) 混合性数据

混合数据类型, 结合非结构化数据和结构化数据两者的特点进行使用。

2.2 风场数据概述及预处理

风场数据的常用的数据存储格式是 NetCDF 格式, 是一种面向数组的适合网格数据的一种跨平台的数据交换格式^[69]。面向数组的 NetCDF 格式数据非常适合大气、洋流等矢量场数据, 因为它的数据量大且结构简单, 适合存储。

2.2.1 NetCDF 概述

NetCDF 是面向数组的数据访问的接口和提供接口实现的库。NetCDF 库还定义一种表示科学数据接口, 用来支持科学数据的创建、访问和共享。NetCDF 软件是在科罗拉多州博尔德的 Unidata 项目中心开发的, 是一种用来存储温度、

湿度、气压、风速和风向等多维科学数据的文件格式。在 ArcGIS 中，通过根据 NetCDF 文件创建图层或表视图。NetCDF 被广泛用于大气科学、海洋学、地球物理、洋流等诸多领域，用户可以借助诸多方式进行管理 and 操作 NetCDF 数据集，包含以下特性：

描述性：包含所需要的数据信息，如经纬度，UV，风力等级等信息。

跨平台性：不依赖于某种操作系统，可通过多种操作系统的软件进行读取。

可选择性：可选择性读大文件中的小子集，仅读取自己所研究的数据文件，提高读取文件的效率。

(2)NetCDF 数据的存储

NetCDF 文件中的数据以数组形式存储。例如：如某个区域内随时间和高度变化的温度，以一系列二维数组的形式存储，如图 6^[70]。

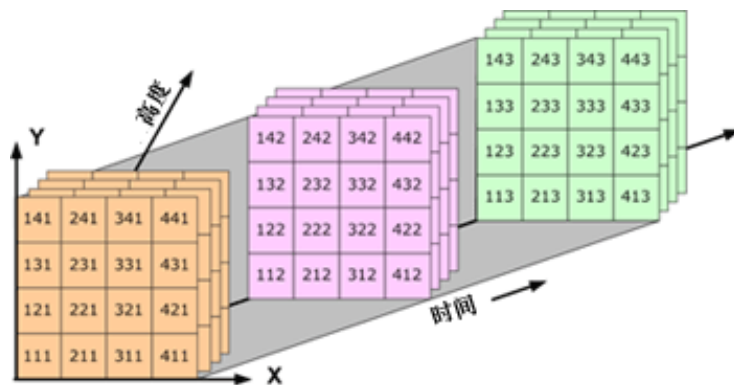


图 6 某个区域随时间和高度变化的数组^[70]

Figure 6 Array of an area that varies with time and height

(3)NetCDF 的基本组成部分

NetCDF 文件包括三种描述类型：维度、属性和变量，这些类型共同描述一个数据集。描述类型又可分为适用于整个文件的全局属性和适用于单个小文件的局部属性，全局属性描述的是数据集的来源和文件格式等基本属性，维度存储格网等级、时间和经纬度信息。将这三者描述类型结合起来使用足以说明面向数组的数据集中各数据字段之间关系的含义和联系。

网络公用数据格式语言(Common Data Language, 简称“CDL”)是用于描述 NetCDF 文件内容的 ASCII 格式^[70]，将变量坐标、全局属性和局部属性结合起来使用可以说明数据以及面向数组的数据集中各数据字段之间关系的含义。图 7 显示的是采用 CDL 表示法来描述的 NetCDF 文件结构。

```

netcdf file:/C:/Users/NXD006/Desktop/demo.nc {
  dimensions:
    lev = 1;
    time = 1;
    lat = 361;
    lon = 720;
  variables:
    float U(lev=1, time=1, lat=361, lon=720);
      :abbreviation = "UGRD";
      :max = 42.99183f; // float
      :min = -54.00817f; // float

    float V(lev=1, time=1, lat=361, lon=720);
      :long_name = "v-component of wind @ Planetary Boundary Layer";
      :abbreviation = "VGRD";
      :coordinates = "reftime time lat lon ";
      :max = 43.885204f; // float
      :min = -44.614796f; // float

    float lat(lat=361);
      :units = "degrees_north";
    double lev(lev=1);
      :long_name = "Level";
    float lon(lon=720);
      :units = "degrees_east";
    double reftime;
      :units = "Hour since 2018-09-16T00:00:00Z";
      :standard_name = "forecast_reference_time";
      :long_name = "GRIB reference time";
      :calendar = "proleptic_gregorian";
    double time(time=1);
      :units = "Hour since 2018-09-16T00:00:00Z";
      :standard_name = "time";
      :long_name = "GRIB forecast or observation time";
      :calendar = "proleptic_gregorian";

    // global attributes:
    :GRIB_table_version = "2,1";
    :Type_of_generating_process = "Forecast";
    :file_format = "GRIB-2";
    :Conventions = "CF-1.6";
}

```

坐标变量

变量属性

全局属性

图 7 CDL 文件结构

Figure 7 Sample CDL file

2.2.2 风场数据预处理

本文实验所需要的风场数据是由 NOAA^[71]的 NCEP（国家环境预测中心）机构建立的 GFS（全球预报系统）天气预报模型生成的风场数据，此模型可生成数十种大气数据和陆地土壤变量的数据，包括温度、风、降水、土壤湿度和大气臭氧浓度。

本文所需要的风场数据可 NOAA 下的模型分发系统网站进行下载，时间范围是 2021 年 12 月 1 日，区域范围是采用经度 0-360°，纬度为 90° N~90° S，大气层水平的全球风力数据，对所获取的数据进行筛选，如图 8。

For GRIB-2 data only.

Select the levels desired:

☐ all ☐ 0-0.1 m below ground ☐ 0.1-0.4 m below ground ☐ 0-2 m below ground ☐ 0.4-1 m below ground ☐ 10 m above ground ☐ 1-2 m below ground ☐ 1 hybrid level ☐ 2 m above ground ☐ boundary layer cloud layer ☐ convective cloud bottom level ☐ convective cloud layer ☐ convective cloud top level ☐ entire atmosphere (considered as a single layer) ☐ high cloud bottom level ☐ high cloud layer ☐ high cloud top level ☐ low cloud bottom level ☐ low cloud layer ☐ low cloud top level ☐ middle cloud bottom level ☐ middle cloud layer ☐ middle cloud top level ☐ surface ☐ top of atmosphere

Select the variables desired:

☐ all ☐ ACOND ☐ ALBDO ☐ CDUVB ☐ CNWAT ☐ CPRAT ☐ CRAIN ☐ CSDLF ☐ CSDSF ☐ CSULF ☐ CSUSF ☐ CWORK ☐ DLWRF ☐ DSWRF ☐ DUVB ☐ EVBS ☐ EVCW ☐ FRICV ☐ GFLUX ☐ HGT ☐ HPBL ☐ ICEC ☐ ICETK ☐ LAND ☐ LHTFL ☐ NBDSF ☐ NDDSF ☐ PEVPR ☐ PRATE ☐ PRES ☐ PWAT ☐ QMAX ☐ QMIN ☐ SBSNO ☐ SFCR ☐ SFEXC ☐ SHTFL ☐ SLTYP ☐ SNOD ☐ SNOHF ☐ SNOWC ☐ SOILL ☐ SOILM ☐ SOILW ☐ SOTYP ☐ SPFH ☐ SRWEQ ☐ SSRUN ☐ TCDC ☐ TMAX ☐ TMIN ☐ TMP ☐ TRANS ☐ UFLX ☐ UGRD ☐ U-GWD ☐ ULWRF ☐ USWRF ☐ VBDSF ☐ VDDSF ☐ VEG ☐ VFLX ☐ VGRD ☐ VGTYP ☐ V-GWD ☐ WATR ☐ WEASD

Extract Subregion

File transfer times can be reduced by only transferring a subregion. You can use this section to extract a geographic subsection from a most GRIB files. Use negative numbers for south and west.

make subregion ☐ left longitude right longitude
top latitude bottom latitude

图 8 多种 NetCDF 数据类型

Figure 8 Multiple NetCDF data types

因为本文所设计的原型系统是关于风场的可视化，因此只需记录一些相关的特定等压在线(大气层水平)的风场数据。使用 Panoply 软件进行查看解析，Panoply 是基于 Java 虚拟机进行开发的 NetCDF 译码器，本文对包含海洋矢量场数据的 NetCDF 文件进行解析，并提取出所需要的风场数据（U 方向）影像，如图 9。

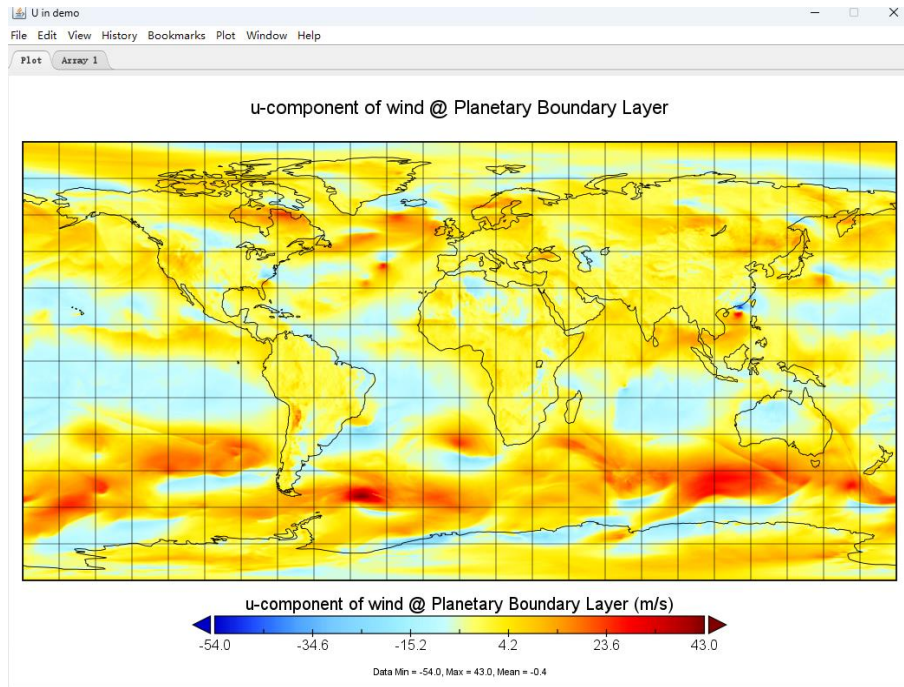


图 9 初始原始风场数据(U 方向)

Figure 9 Initial raw wind farm data (U direction)

将本文获取的数据要通过设置代理服务的形式进行发布和管理，通过 JavaServlets 设计代理接口，同时对于代理管理的数据要进行处理一定格式转换等操作，将数据转换成 Cesium 所能识别的数据格式，通过 netcdjs 类库对风场数据格式从 nc 转换成 json 格式。对于解析出来的风场数据集合如图 10，并不能全部符合本文所要求的条件，因此要进行数据筛选处理操作，通过 Panoply 软件对于不符合渲染、属性为空和位置不符合要求的数据点要进行删除操作，最后经筛选完的形成的部分数据集，结果如图 11。

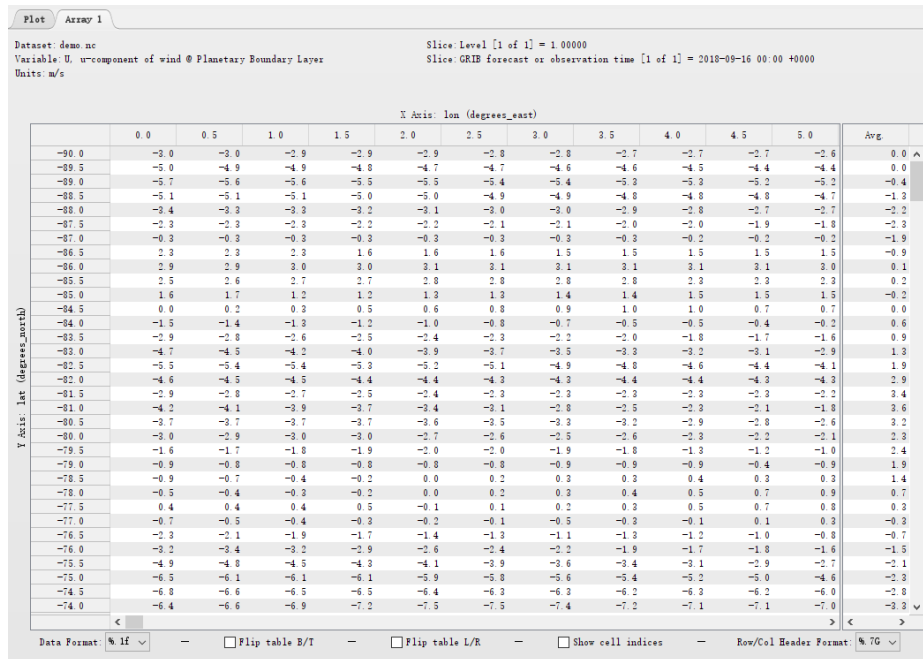


图 10 解析数据集

Figure 10 Parsing nc file

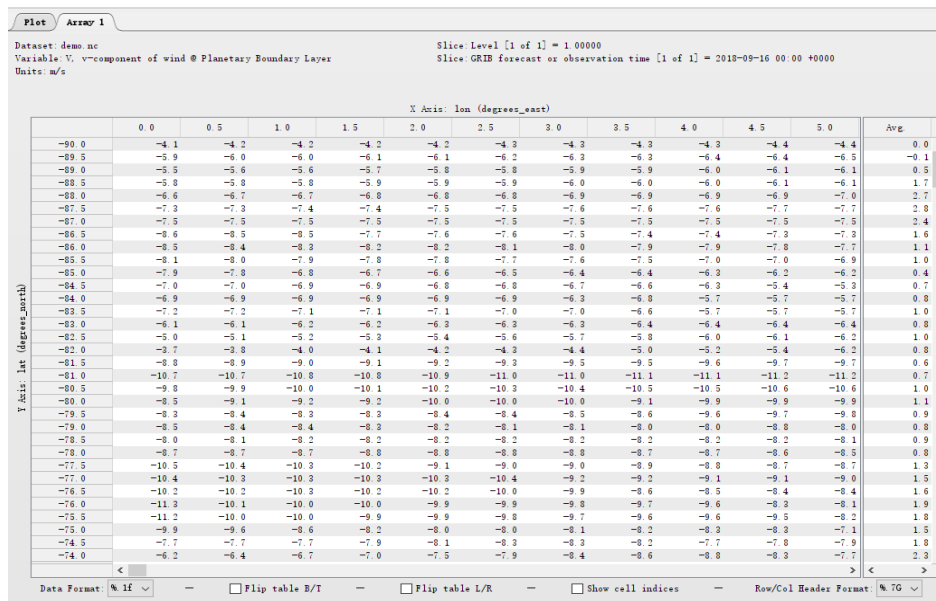


图 11 筛选数据集

Figure 11 The parsed data set

综上所述，对于本文所处理的 NetCDF 文件预理解析的流程，如图 12：

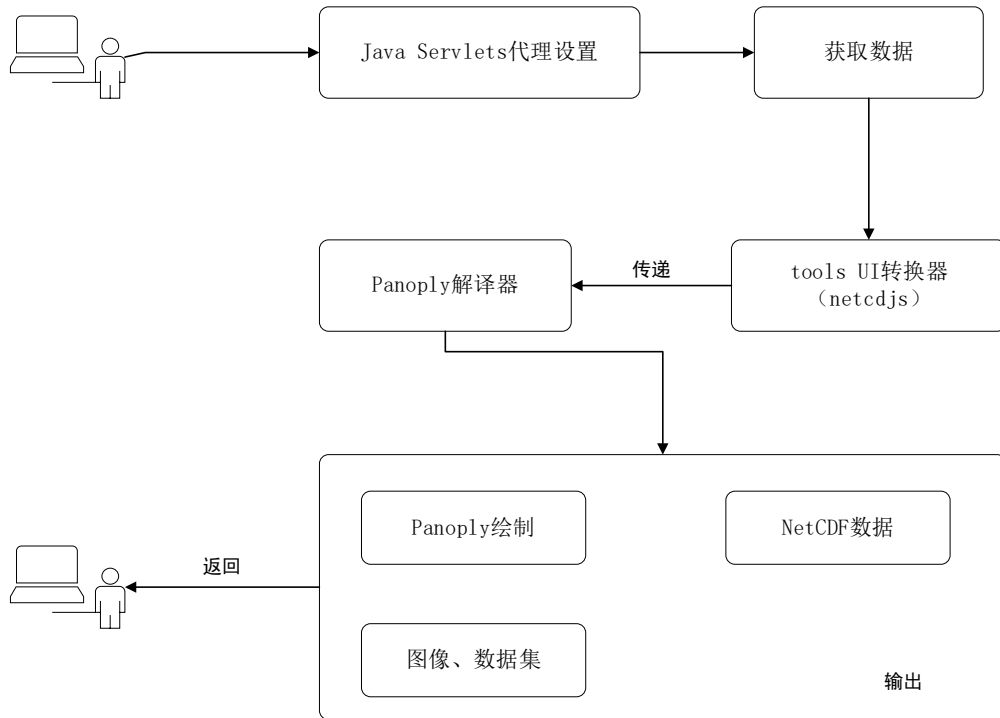


图 12 NetCDF 数据处理流程
Figure 12 NC data processing flow

2.3 本章小结

本章是关于 NC 数据的解析。关于数据对象，首先介绍矢量场数据及其矢量场数据的结构特点、数据类型等。对于本文所需要的风场数据格式，首先描述 NetCDF 数据的结构、存储以及数据文件的格式、获取方式以及关于 NetCDF 数据的数据解析、筛选和格式转换、数据代理等预处理操作，并根据以上流程将所得到的矢量场数据转化为本文研究所需要的数据格式进行数据的使用和渲染，以供下一步的工作使用。

3 基于粒子系统的可视化方法研究

矢量场的可视化是人们借助先进的计算机图形学技术，通过计算机模拟仿真，从而形成一个新的学科方向。

3.1 矢量场数据动态可视化的主要方法

3.1.1 点图标法

点图标法虽然绘制简单，但存在较多的缺陷。特别是当面对大量数据的绘制时，点图标法存在很大的缺陷，数据密集，容易导致图像表达不准确，无法准确的表现出风向；数据过于稀疏，又不能准确把握矢量场的变化情况，很难表达数据的特点。这种方法多适用于简单的矢量场，但涉及到复杂场景无法揭示出数据的内在连续性，风场中的一些重要特征像涡流等结构也很难用点图标的方式表达清楚，如图 13。

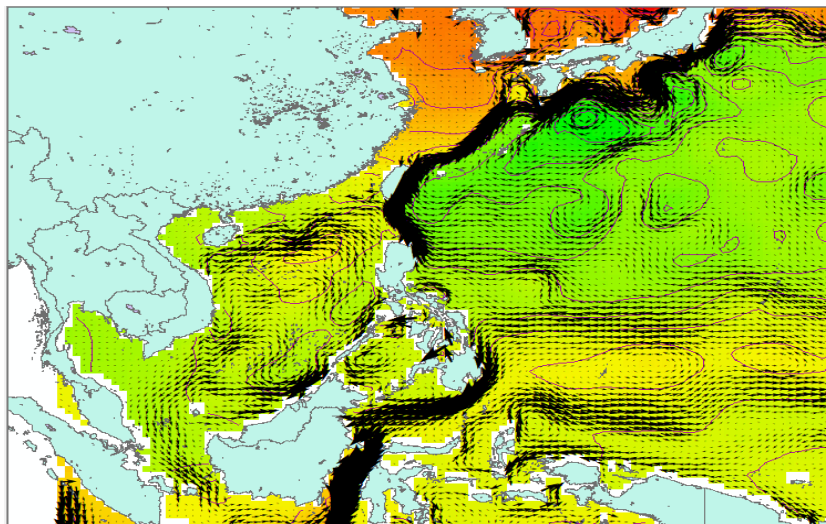


图 13 点图标法

Figure 13 Point Icon Method

3.1.2 矢量线法

矢量线方法源于实验型流场的可视化技术。矢量线方法包括迹线、流线、迹线和时线等。流线一般是用于稳定矢量场，其它形式的矢量线方法一般用于非稳定风场。与点图标法相比，矢量线可在一定程度上表示出矢量场的连续性，绘制效果也有较大的改善，但可视化图像的绘制质量严重依赖于种子点的位置的选取，并且其表示效果是静态的，无法动态的反应矢量场数据的动态变化情况。同时当种子点数据取得较密时，同样会造成视觉上的混乱。

3.1.3 特征法

特征可视化就是通过对矢量场中的这些特征，减少可视化映射的数据量，特征可视化是矢量场可视化研究领域面对大数据量的热点方法之一。但是此

方法过于依赖领域的选择。其中此方法的矢量场特征主要包括两种，其中一种是从大量数据中分离出来的用户感兴趣的区域，第二部分是具有一些特殊现象的结构特征形状，现象和变化，例如激流、台风眼、涡流等矢量场。保持流场特有的流场现象。在特征可视化过程中，可得到一种可代替原始数据的抽象可视化表示，使用户自动里分出一些自己感兴趣的区域。

3.1.4 纹理映射方法

纹理映射法是通过纹理的形式进行表达，纹理是按照排列组合形成的颜色图，纹理有两种属性，它们是颜色和形状，因此纹理映射法在矢量场可视化中具有一定的优势。纹理映射具有图像空间的连续性，可生成具有图像空间分辨率的细致图形。基于纹理的可视化图像表现出一定的几何形状，通过颜色的有序排列，表达出一定的方向信息。基于纹理的映射方法，综合颜色映射方法和几何形状映射方法两者的优势，有效避免两者的劣势，如图 14，此方法为线积分卷积法(LIC)。但此方法计算过于复杂，实现过程也较为困难，同时用卷积来表示矢量场的方向是一种运动模糊的思想，对于非专业的研究人员，此方法过于复杂化，不宜实现。

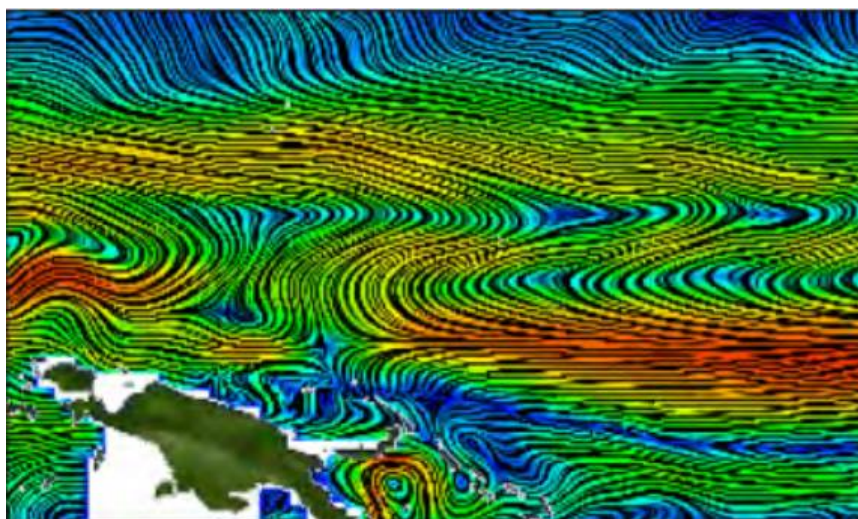


图 14 LIC 方法效果图^[72]

Figure 14 Effect of LIC method

3.1.5 粒子系统法

粒子方法是利用光照、颜色等信息进行可视化，可用于模糊对象的造型和绘制，能表示出火、树等不规则的复杂几何体，也可应用于流场、风场、波浪等矢量场可视化中。

粒子系统的最小单元是粒子，是粒子的集合体，每一个不同的粒子都有属于它们自己的生命周期。粒子系统的生命周期分为“出生”、“活动”和“死亡”三个阶段，如图 15。在粒子的生命周期中，粒子的各种性质，比如形状、颜色、大小、透明度等视觉性质和位置、速度、运动方向角度等动态性质，都会随着时间的改变而改变，同时可将粒子的具体某一性质和矢量联系起来，建立映射关系。

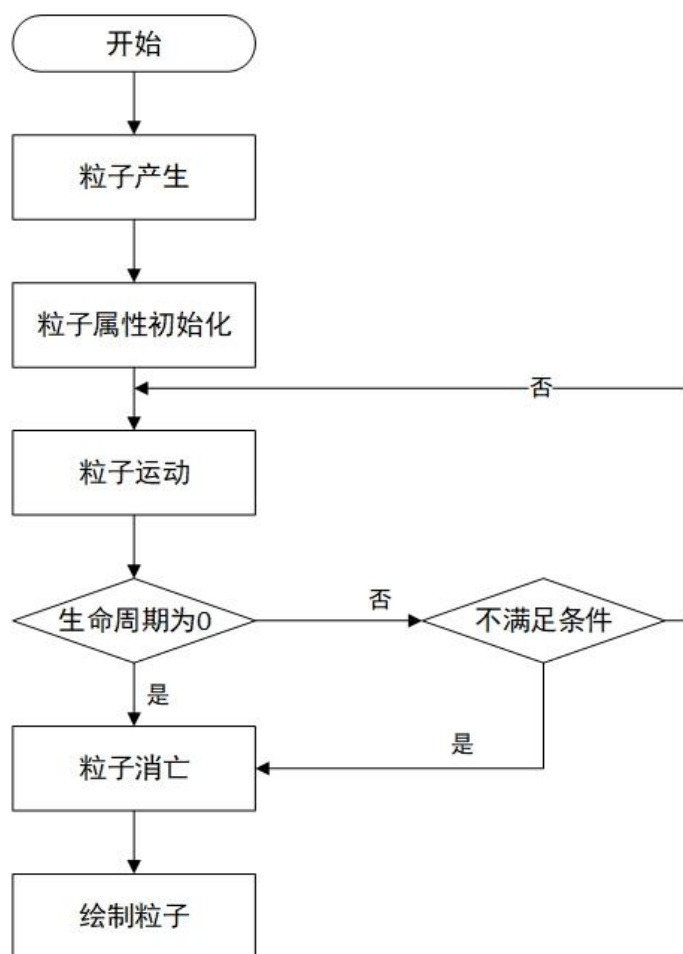


图 15 粒子系统流程图

Figure 15 Flow chart of particle system

综合以上可视化方法，在实现矢量场动态可视化效果和实现难易程度的综合考虑下，粒子方法是最为合理且理想的方法，在矢量场可视化中将粒子的动态性质和视觉性质和矢量场中的矢量数据联系起来，能够得到较为理想的可视化效果^[74]。对风场的轨迹显示可采取动画粒子的形式进行可视化，根据粒子所处位置的不同从而直接表达风场的速度变化^[75]。

粒子系统方法是一种基于物理模型解决问题的方法，核心是选取合适的物理模型，物理模型的复杂程度决定粒子的运动规律和复杂程度。因此针对粒子系统的粒子追踪算法选择合适的物理模型并进行改进模型，使粒子的运动轨迹更加符合实际情况，达到逼真的矢量场可视化效果极其重要。

3.2 基于粒子系统的风场数据动态可视化算法改进

本文应用的是粒子移动技术是数值矢量场中的模拟技术^[76]，粒子可以看作是可视化场景的运动质点，粒子的不同轨迹可看作一条条不同长度的线，轨迹是由该点在可视化场景中连续移动的位置组成而成。

根据粒子运动微分方程

$$\frac{dS}{dh} = W(S) \quad (3-1)$$

粒子移动的过程就是运动质点在空间中连续位置变化的过程。从初始点位置开始，在可视化场景中进行移动，具体步骤为：

步骤 1，根据 NetCDF 数据确定空间 S 的空间位置；

步骤 2，在经纬度格网中，通过下文介绍的双线性插值算法求解该点的矢量值和附近未知点的矢量值；

步骤 3，解粒子运动微分方程，求出下一粒子位置。

根据粒子方程给出具体的微分方程的解是

$$y(h + \Delta h) = S(h) + \int W(S(h))dh \quad (3-2)$$

其中 Δh 的选择至关重要，影响着上述微分方程的计算量和误差。

对于此微分方程的最简单的解法就是欧拉法，对于此微分方程的积分项可变成

$$\int W(S(h))dh = W(X(h)) \cdot \Delta h \quad (3-3)$$

因此微分方程可变为：

$$y(h + \Delta h) = S(h) + W(X(h)) \cdot \Delta h \quad (3-4)$$

对于欧拉法的解放，用泰勒展开式得到的精度为 $O(t^2)$ ，误差比较大，因此欧拉法的逼近方式是相当不理想的，对数据的中心会产生较大的影响，比如漂移现象，因此欧拉法是不符合要求的。

考虑到数据中心的偏移现象的影响，因此本文想选择精度更高的四阶龙格库塔算法以此来解决数据中心的偏移现象的影响，进行对粒子进行跟踪将粒子的轨迹绘制出来以此来表示风场的轨迹和运动趋势。

3.2.1 双线性插值算法原理

在地理数据处理过程中，获取的风场（洋流、降雨量等）矢量场数据是各个采集点采集的数据，这些数据低素质不均匀分布在各个地区，无法保证每个地区都有相对应的数据，若该地区没有相对应的风场数据就要通过附近的采集点进行粗略估算所需位置（当前位置）的风场数据，因此就要用到插值算法进行处理数据插值问题，插值算法主要用于处理图像处理，主要应用于图像的缩放等问题，但是随着数据的需求，插值算法也逐渐用于 GIS 的数据处理等问题，用来估算未知点的数据。在积分之前，要解决一部分数据缺失等问题，因此要采用数据插值的方法，将一些所需要的数据补充完整，保证场数据的完整，效果细腻。

现在比较常用的是插值算法是最邻近插值和双线性插值，双三次立方插值法也应用的比较广泛。最邻近插值算法是搜索当前坐标点最邻近的数据点，取该点的数据值作为当前点的数据值，可通过求取欧式距离的快速定位方法定位当前坐标点最邻近的数据点的坐标，获取待求点的数据。它虽计算简单，采样速度快，但缺点过于明显，数据跳跃性显著，计算精度低，数据结果受位置的影响比较大，对于图像处理，缩放质量差，会导致图片失真。三次立方卷积法的数据处理准确，位置精度高，但计算量过大，计算速度慢。相比较以上两种插值算法的优缺点，双线性插值算法的计算量适中，对于非专业人员比较友好，同时对于求解的位置点的矢量值精度高。因此本文采用双线性插值，进行处理数据不显示，数据稀疏、图像缩放导致坐标点发生改变等情况。

在数学上，双线性插值是对线性插值在二维直角网格上的扩展，用于对双

变量函数（例如 U 和 V ）进行插值。其核心思想是在两个方向分别进行一次线性插值。根据最邻近的多个数据点插值计算当前未知数据点的数值，此算法主要用来处理网格数据，根据待求坐标点所在网格的四个附近的格网点的数值，结合位置距离加权计算得到待求坐标点的数值，如图 16。

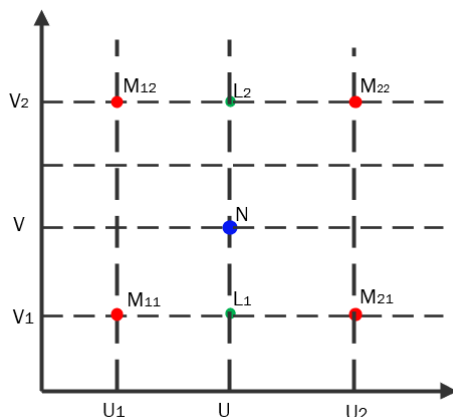


图 16 线性插值原理

Figure 16 Principle of Linear Interpolation

从图中可以看出，从 M_{11} ， M_{12} ， M_{22} ， M_{21} 四个已知的红色数据点和待插值的 N 蓝色的未知数据点。此时想通过未知函数 $y(u, v)$ 在 N 点 $N = (u, v)$ 的值。假设此时已知 $y(u, v)$ 在 $M_{11} = (u_1, v_1)$ ， $M_{12} = (u_1, v_2)$ ， $M_{22} = (u_2, v_2)$ ， $M_{21} = (u_2, v_1)$ 四个点的值。

首先，第一步在 x 方向进行一次线性插值，得到：

$$y(u, v_1) \approx \frac{u_2 - u}{u_2 - u_1} y(M_{11}) + \frac{u - u_1}{u_2 - u_1} y(M_{21}) \quad (3-5)$$

$$y(u, v_2) \approx \frac{u_2 - u}{u_2 - u_1} y(M_{12}) + \frac{u - u_1}{u_2 - u_1} y(M_{22}) \quad (3-6)$$

第二步是在 y 方向进行同样的一次线性插值，得到：

$$y(u, v) \approx \frac{v_2 - v}{v_2 - v_1} y(u, v_1) + \frac{v - v_1}{v_2 - v_1} y(u, v_2) \quad (3-7)$$

最后计算出 $N(x, y)$ 处的值，为：

$$\begin{aligned} y(u, v) \approx & \frac{v_2 - v}{v_2 - v_1} \left(\frac{u_2 - u}{u_2 - u_1} y(M_{11}) + \frac{u - u_1}{u_2 - u_1} y(M_{21}) \right) \\ & + \frac{v - v_1}{v_2 - v_1} \left(\frac{u_2 - u}{u_2 - u_1} y(M_{12}) + \frac{u - u_1}{u_2 - u_1} y(M_{22}) \right) \end{aligned} \quad (3-8)$$

令 $y(u, v) = N(u, v)$ ，此时对 N 点继续合并拆分等操作，即可推导为：

$$\begin{aligned}
N(x,y) &= \frac{1}{(v_2-v_1)(u_2-u_1)} [y(M_{11})(v_2-v)(u_2-u) + f(M_{22})(v-v_1)(u-u_1) \\
&\quad + f(M_{21})(v_2-v)(u-u_1) + y(M_{12})(v-v_1)(u_2-u)] \\
&= \frac{1}{(v_2-v_1)(u_2-u_1)} \begin{bmatrix} u_2-u & u-u_1 \end{bmatrix} \begin{bmatrix} y(M_{11}) & y(M_{12}) \\ y(M_{21}) & y(M_{22}) \end{bmatrix} \begin{bmatrix} v_2-v \\ v-v_1 \end{bmatrix}
\end{aligned} \quad (3-9)$$

即：

$$y(u,v) \approx \frac{1}{(v_2-v_1)(u_2-u_1)} \begin{bmatrix} u_2-u & u-u_1 \end{bmatrix} \begin{bmatrix} y(M_{11}) & y(M_{12}) \\ y(M_{21}) & y(M_{22}) \end{bmatrix} \begin{bmatrix} v_2-v \\ v-v_1 \end{bmatrix} \quad (3-10)$$

通过以上介绍的插值方法，可得到经纬格网坐标内的函数值，其它点未知待求点位置也可通过双线性插值算法求得。

本文基于三维的风场数据进行双线性插值，不考虑 z 轴方向的影响，仅考虑 U 和 V 轴。按照 NC 数据的特点和经纬度的特点，将 U, V 分别沿着经度方向和纬度方向投影，在计算过程中只需要计算经度和纬度方向的风力矢量分量，然后分别计算重新生成的粒子需要在经度和纬度方向的移动速度，最后更新粒子的坐标位置，然后绘制粒子运动的渲染轨迹图，如图 17。

粒子绘制具体步骤，

- 步骤 A，生成自定义粒子，初始化粒子状态；
- 步骤 B，根据风场数据的坐标，获取相对应的粒子坐标；
- 步骤 C，计算粒子的矢量场 UV 坐标；
- 步骤 D，插值计算新的粒子坐标；
- 步骤 E，更新粒子坐标；
- 步骤 F，绘制粒子轨迹。

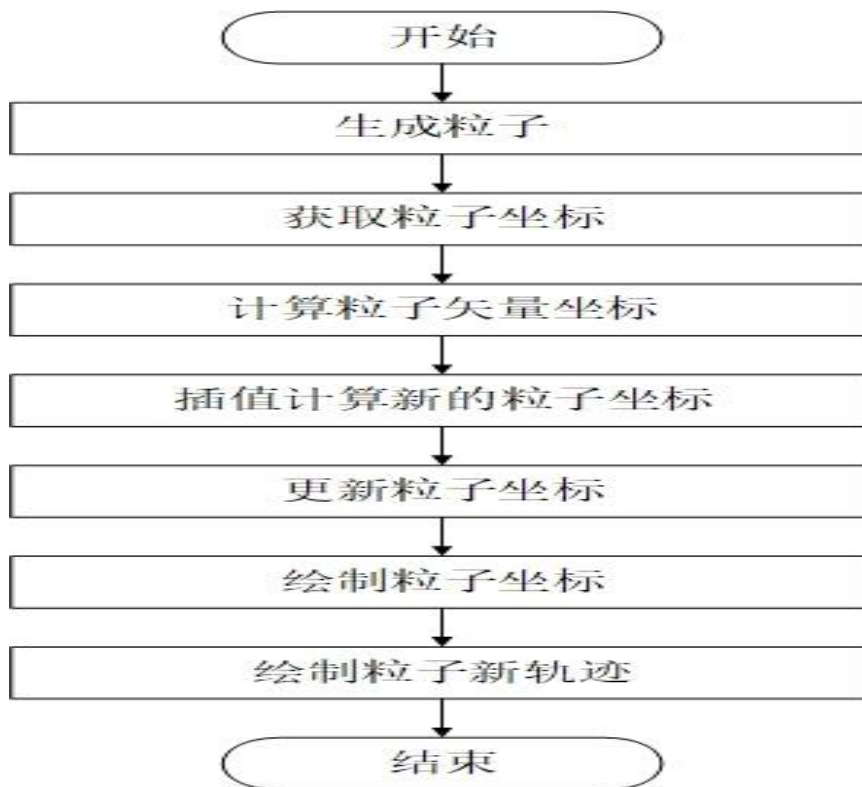


图 17 粒子更新流程图

Figure 17 Flow chart of particle update

3.2.2 四阶龙格-库塔算法原理

在数值分析中,用于非线性常微分方程的解法的常用的一种显式或隐式迭代法就是龙格-库塔算法(Runge-Kutta Methods)。泰勒展开式和通过斜率近似表达微分方程的理论基础来源于龙格-库塔算法^[77]。

根据泰勒展开式和微分方程,对应的龙格库塔算法有各种不同的解法,但是精度较高的就是四阶龙格-库塔算法(简称“RK4 或者龙格库塔算法”)。四阶龙格-库塔算法是已知初始值和方程导数等信息,利用计算机的仿真应用,省去求解微分方程的复杂过程。四阶龙格-库塔算法的计算步骤为,

步骤 1, 求解各分段积分区间的相关点的斜率

步骤 2, 将步骤 1 计算得到的各个积分区间的斜率进行加权平均计算,

步骤 3, 将步骤 2 中的计算得到的加权值用于表示下一个点的计算, 计算过程如下, 令初值表达如下,

$$\begin{cases} y' = f(t, y) \\ y(t_0) = y_0 \end{cases} \quad (3-11)$$

则, 对于该微分方程问题的解有多个, 其中四阶龙格-库塔(RK4) 是一个较优解, 四阶龙格-库塔算法给出一下方程:

$$f(t)_{n+1} = f(t)_n + \frac{h}{6}(k_1 + 2k_2 + 3k_3 + k_4) \quad (3-12)$$

计算出积分区间的四个点的斜率, 其中

$$\begin{aligned} k_1 &= f(t_n, y_n) \\ k_2 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \\ k_3 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \\ k_4 &= f(t_n + h, y_n + hk_3) \end{aligned} \quad (3-13)$$

这样, 下一个值 y_{n+1} 由求得的值 y_n 加上时间间隔 (h) 和一个加权计算得

到的斜率 k 的乘积做决定。该斜率是以下斜率的加权平均:

k_1 ——时间段开始时的斜率;

k_2 ——时间段中点的斜率(第一段和第二段时间的连接处); 通过欧拉法采用斜率 k_1 来决定 y 在 $t_n + \frac{h}{2}$ 的值;

k_3 ——时间段的中点斜率(第二段和第三段时间的连接处), 但是这次采用斜率 k_2 决定 y 值;

k_4 ——时间段终点的斜率, 终点的斜率有更大的权值;

3.2.3 基于 WebGL 的变步长的龙格—库塔算法

Buning^[78]认为数值积分的阶数、矢量场插值、时间步长、积分顺序对跟踪算法产生的误差。四阶龙格-库塔算法的精度误差比较精细，可以达到 $O(t^5)$ ，用此方法绘制粒子的轨迹比较方便，但存在一定的误差，特别是涉及到数据密集和稀疏的地方，尤其是风场数据的速度变化比较快，步长 h 的大小关系到每一步的计算误差，要尽量避免因步长 h 的截断误差和计算误差对本文的实验结果产生不可预测的误差，需要考虑这些因素。

因此时间步长的大小会对局部误差产生影响，风场数据有 U 、 V 两个方向的矢量，首先是从 U 方向进行解析，对一阶常微分方程式的初始值问题

$$\begin{aligned} y' &= f(t, y) \\ y(t_0) &= y_0 \end{aligned} \quad (3-14)$$

对于求解初值问题时，求解出它的解在某些节点函数值的近似值。假定 y_n 为 $y(t_0)$ 的近似解，其中 $t_n = t_{n-1} + h_{n-1}, n=1, 2, \dots$ 。Runge-Kutta 算法在计算 y_{n+1} 时只利用 y_n 。因此对于式(3-12)的单步计算公式可变为

$$f(t)_{n+1} - f(t)_n = h_n \phi(t_n, f(t)_n, h_n) \quad (3-15)$$

h_n ——时间间隔

$\phi(t_n, f(t)_n, h_n)$ ——为此时估算的斜率。

假定 $\bar{f}(t)$ 为公式 $d\bar{y}/dt = f(t, \bar{y})$ 用 $\bar{f}(t_n) = f(t)_n$ 初始条件的解，因此可将误差定义为

$$d_{n+1} = \bar{f}(t)_{n+1} - f(t)_n - h_n \phi(t_n, f(t)_n, h_n) \quad (3-16)$$

从上式可以看出局部误差的精确程度跟步长 h 有关。因此控制步长对于本文的可视化效果应有一定的改善，原始算法采用的是固定步长，但固定步长适合一些数据分布比较均匀的地方，对数据分布不均匀的地方，即偏转角变化比较大地方，固定步长的积分算法会对实验结果造成一定的影响，会对风场的中心造成一定的视觉混淆效果，不符合要求的效果，因此在积分求解的过程中要改变积分步长的长度，使得误差变得更小，使风场的轨迹更加符合效果，保证较高的精度。

在变步长的龙格库塔算法中，特别是涉及到风场比较复杂和数据密集复杂区域，对于此问题的解决本文采用改变积分步长的动态变化方法，调整积分步长的大小，对于式(2-13)，从节点 t_n 出发，首先以步长 h 求解出一个近似值

$y_{n+1}^{(h)}$ ，由于公式的截断误差为 $O(h^5)$ ，进而求出

$$f(t_{n+1}) - f^{(h)}_{n+1} \approx o(h^5) \quad (3-17)$$

然后将步长折半，即取 $\frac{h}{2}$ 为步长从 t_n 前进两步到 t_{n+1} ，进而在求得一个近似值

$f(t)_{n+1}^{h/2}$ ，每前进一步得到的截断误差是 $o(\frac{h}{2})^5$ ，因此有

$$f(t_{n+1}) - f(t)_{n+1}^{(\frac{h}{2})} \approx 2o(\frac{h}{2})^5 = \frac{1}{16}o(h)^5 \quad (3-18)$$

通过比较以上两式可以得知，当将步长减半后，我们所得的误差大约可少到 $\frac{1}{16}$ ，

即是

$$\frac{f(t_{n+1}) - f(t)_{n+1}^{(\frac{h}{2})}}{f(t_{n+1}) - f(t)_{n+1}^h} \approx \frac{1}{16} \quad (3-19)$$

转换上式可得

$$f(t_{n+1}) - f(t)_{n+1}^{(\frac{h}{2})} \approx \frac{1}{15} \left[f(t)_{n+1}^{(\frac{h}{2})} - f(t)_{n+1}^{(h)} \right] \quad (3-20)$$

因此可以通过检查步长，将两次计算结果得到的偏差记为 Δ ，即

$$\Delta = \left| f(t)_{n+1}^{(\frac{h}{2})} - f(t)_{n+1}^{(h)} \right| \quad (3-21)$$

来进行判断所选的步长是否合适。

综上所述，具体实现步骤如下，

步骤 1，依据粒子轨迹线的前后两个夹角大小来决定步长是否变化；

步骤 2，计算折半前后两次的偏差 $\Delta = \left| f(t)_{n+1}^{(\frac{h}{2})} - f(t)_{n+1}^{(h)} \right|$ 来判断所选的步长是

否合适；

步骤 3，对于矢量偏转角变化大的地方，数据点稀疏的地方需要对给定的步长进行缩小；

步骤 4，对于给定的精度 ε ，如果对于 $\Delta > \varepsilon$ ，重复步骤 3 将步长进行折半计算 $f(t)_{n+1}^h$ ，直至 $\Delta < \varepsilon$ 为止，将此时的步长减半进行减半作为最佳步长进行输

出，取最终的 $f(t)_{n+1}^{(\frac{h}{2})}$ 作为结果；并将求得的计算结果代入到式(2-11)中，即

$$f(t)_{n+1} = f(t)_n + \frac{f(t)_{n+1}^{(\frac{h}{2})}}{6} (k_1 + 2k_2 + 3k_3 + k_4) \quad (3-22)$$

步骤 5，涉及到数据比较稀疏的地方，对于矢量偏转角变化小的地方，需要对给定的进行缩小；

步骤 6，如果 $\Delta < \varepsilon$ ，重复步骤 5 将步长进行增倍计算，直至 $\Delta > \varepsilon$ 为止，将

此时的步长作为最佳步长输出，取 $f(t)_{n+1}^h$ 作为最终结果；并将求得的计算结果代入到式(2-11)中，即

$$f(t)_{n+1} = f(t)_n + \frac{f(t)_{n+1}^h}{6}(k_1 + 2k_2 + 3k_3 + k_4) \quad (3-23)$$

操作以上步骤，龙格库塔的步长进行增半减半的操作，并将最后的步长加入到四阶龙格库塔算法再根据 NC 数据中的数据坐标分量可以较为准确计算出粒子的当前位置点和下一位置的精确位置的 U 分量；对于 V 方向产生的截断误差也是按照 U 方向的计算过程进行计算，两者的计算步骤大致相同只不过就是方向不一样而已，只不过最后要将两者的计算结果通过三角函数进行合并，计算出我们索要求得的粒子位置点。

尽量使绘制的粒子轨迹符合实际情况，效果展示更加细腻，具体算法实现过程可从附录 1 得知。

3.2.4 基于颜色映射的风场数据可视化

颜色映射算法是图形可视化中应用最广泛的算法，此算法的核心是建立数据属性和颜色属性之间的映射关系，用相对应的颜色渲染相关的数据。本文所设计的可视化原型系统是通过粒子的颜色、大小、形状等属性表达，可直观的分析风场的方向、压强等信息，。在更新粒子坐标，绘制粒子的运动曲线轨迹的同时，更新粒子颜色。

本文为将风场的风速压强等属性通过流线的形式直观的表达出来，采用 RGB 颜色映射表，如图 18。将风速映射到颜色编码的 R 和 G 分量，创建显示用的颜色数据并存放到纹理中，根据粒子创建存储粒子索引的数据并缓冲，最后根据顶点着色器中的数据点的位置和片元着色器中的纹理绘制得到一个对应的粒子颜色，最后颜色随着粒子的生命周期结束而消失，如图 19。

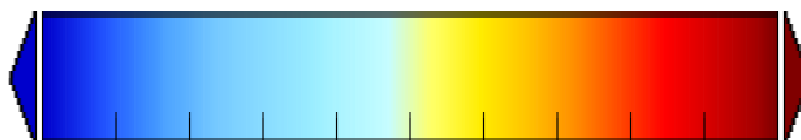


图 18 颜色映射表

Figure 18 Color Map



图 19 粒子颜色初始化

Figure 19 Particle color initialization

在本文中的研究中，对不同的粒子赋予不同的颜色值，建立粒子数量和颜色意义映射关系，将风速映射到不同的粒子中，通过不同的颜色值表示风场的风速和压强等信息，颜色越深表示风速越大；颜色越浅表示风速越低。通过颜色信息我们也可以清晰表达出在全球场景中的风速关系及其运动趋势，有助于我们更好的理解风场运动规律，让本文设计的原型系统更加具有现实意义，也让我们更好的分析运动规律及其对生活的影响。

3.3 实验结果和分析

本文是基于四阶龙格库塔算法对粒子进行跟踪，同时对于初始粒子点的密集程度不同，本文提出对龙格库塔算法的步长根据偏转角的大小不断调整步长的四阶龙格库塔算法，同时对于数据处理采用双线性插值算法，提高数据的分辨率和更新缩放时产生的新坐标，根据实验结果可知这两个算法应用在系统中，取得良好的效果。

对于数据处理的双线性插值算法主要的目的是为解决数据分布不均匀和旋转缩放后产生的坐标变化问题，对于一些数据点不能满足要求的要进行筛选和处理，提高数据的精细化处理精度。

对于变步长的龙格库塔算法的主要目的是解决粒子的轨迹问题，特别是对于矢量偏转角偏大或者偏小的地方，通过不断调整步长，以此来解决轨迹过于直上直下，涡流中心出现漂移等问题，使得风场漩涡中心风场效果变得比较细腻柔和。

3.3.1 原始算法可视化效果

如图 20 的整体效果，可看出四阶龙格库塔算法是一种良好的可视化方法，对于本文的风场展示效果良好，可以看出整体的运动趋势，是符合我们所要求

得整体运动效果。

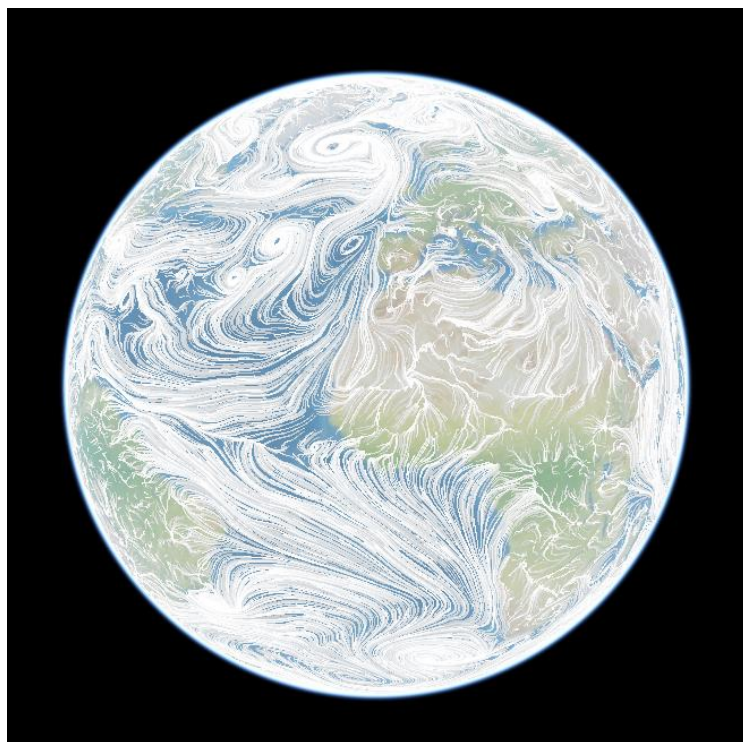


图 20 初步实现效果

Figure 20 shows the initial implementation

当镜头拉近到渲染风场旋涡中心时,展示效果会发生变化,如图 21 采用固定步长的效果图,从图中的红色方框中可以看出,红色方框的旋涡中心轨迹的展示效果不好,轨迹过于直上直下,偏折角度过低,出现旋涡偏移现象,展示效果不合理,导致视觉效果不是很好,不能达到所要预期的效果。因此要对原始的四阶龙格库塔算法进行改进。

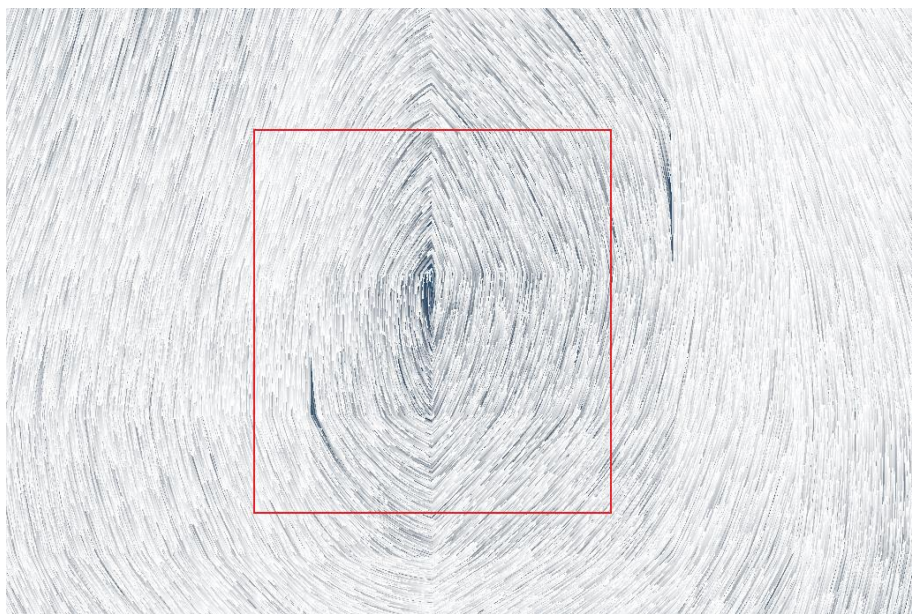


图 21 采用固定步长的效果图

Figure 21 The effect of using a fixed step size

3.3.2 算法改进可视化效果

图 22 变步长的效果图，图中的红色方框是采用本文所改进步长的龙格库塔算法呈现的效果，从图中可以看到风场的旋涡中心的轨迹偏转角适中，轨迹分布比较均匀，展示效果比较合理，并且也符合我们所要要求的效果，从图中我们可以看出本文所改进的算法可以使整个风场效果图更加细腻，尤其是涉及到数据密集的区域，整个风场旋涡展示的效果令人满意。从改进风场漩涡效果可知，本文改进的可视化算法具有一定的创新，并应用到实际场景中，具有一定的实用性和创新性。

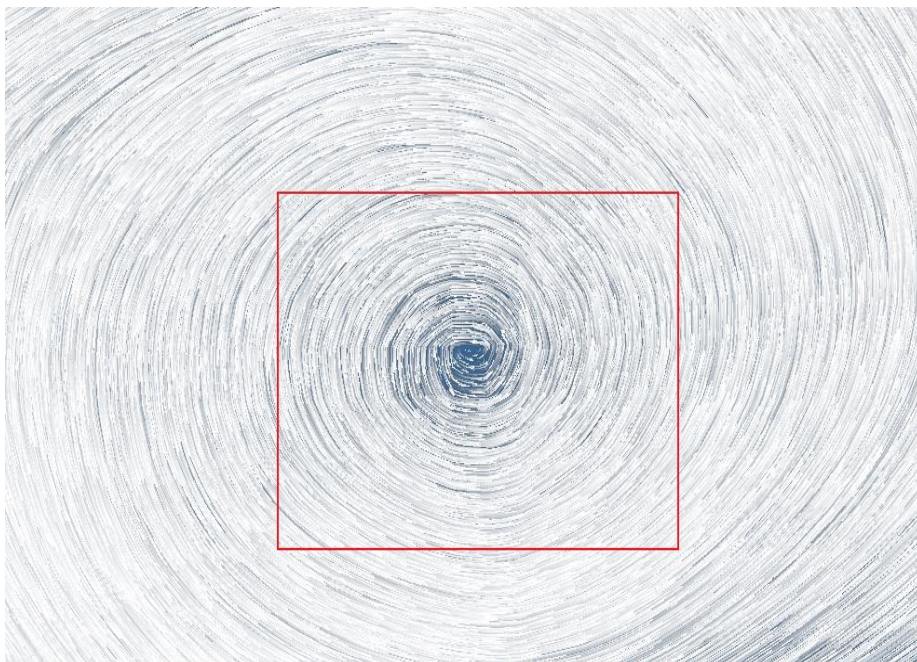


图 22 变步长的效果图

Figure 22 Improved renderings

3.3.3 颜色映射效果

根据颜色与风速的关系，本文对粒子加以颜色映射，从而使整个风场效果显的更加真实有效，也有利于分析风场运动等操作，从而使本文设计的原型系统展示更加真实，具有一定的可用性和真实性，能够表达出更多的风场属性信息和效果，并对现实具有一定的现实意义。图 23 是加以颜色映射的效果，从图中可以看出，对风速不同的地方赋予不同的颜色，可使本文的风场展示效果更好，通过颜色的深浅程度不同和粒子的聚集程度不同，表现出风速和压强的不同。

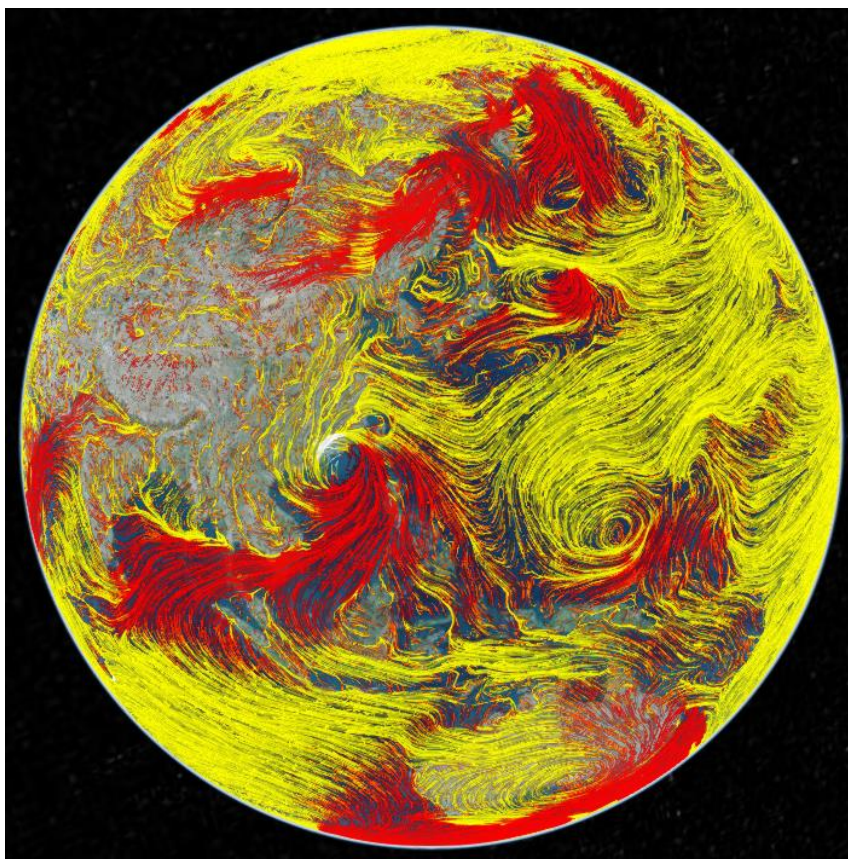


图 23 颜色映射的效果图

Figure 23 Rendering of color mapping

3.4 本章小结

本章主要包含可视化表达形式的选择工作和可视化的算法原理解析工作。

第一部分是关于可视化表达形式的选择。首先介绍五种常见的矢量场数据可视化表达方法，依次比较它们的优缺点，从表现形式、计算量多少、适用条件和实现难易程度等多方面角度综合考虑，选择最为合适的粒子系统的可视化表达方式进行风场的可视化表达。

第二部分是基于粒子系统对风场进行表达。首先是通过介绍基于三维的双线性插值算法，不考虑空间轴即 Z 轴，该算法主要是用于扩展数据集，求解一些数据点的未知矢量值，更新粒子坐标，旋转缩放移动地球时特别需要此算法进行更新粒子的新坐标。对于四阶龙格库塔算法，通过此方法实现对粒子实现跟踪，进而模拟风场的轨迹，对于偏转角过激和速度过快的地方，本文提出通过基于 GPU 调整龙格库塔算法的步长，根据数据的密集程度调整步长，以此提高粒子运动的视觉效果，使得粒子运动轨迹更加符合风场运动效果。最后对粒子进行 RGB 颜色线性映射，通过颜色的不同表示风速的不同，形成一定的颜色视觉反差使得风场的表现效果更加细腻，让实验结果取得良好的效果。

4 基于全球风场可视化场景的渲染加速方法

在浏览器进行全球场景的风场渲染时，要做到实时更新每个粒子的状态，并要及时计算下一步状态，这些大量计算过程对浏览器的负荷较大，对于 CPU 的性能影响非常大，因此要减轻 CPU 的运行压力。GPU 是专门用于处理图像的处理器，但 GPU 只适合简单的线性运算，因此将 CPU 和 GPU 两者同时结合起来处理本文涉及到的计算过程，以此来提高整个风场的运行效率。对于这个合理解决搭配 CPU 和 GPU 问题，本文通过使用 Web Worker 多线程和基于 GPU 创建多帧缓存区加速算法对整个全球风场可视化场景进行渲染加速。

4.1 WebWorker 多线程

运行在旧版浏览器之前的 JavaScript 引擎都是以同步、单线程的进行解析 JavaScript 代码和数据，通过浏览器引擎、JavaScript 引擎、CSS 引擎和呈现引擎实现效果的表达，如图 24，。

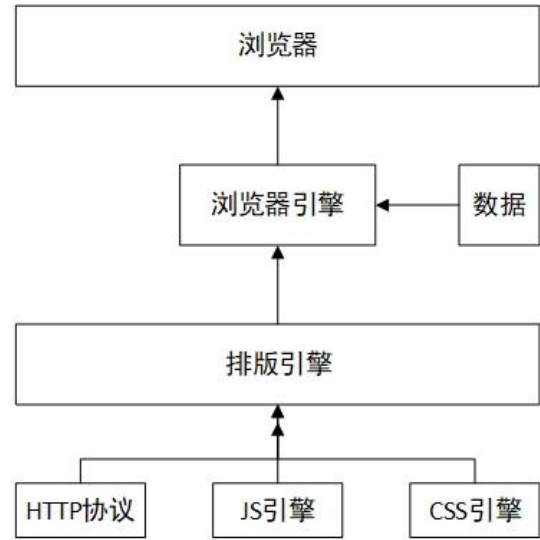


图 24 JavaScript 运行原理

Figure24 How JavaScript works

本文是采用 JavaScript 语言进行编写,由于是在浏览器上运行的标准，效率较差，但随着电脑计算能力的增强，尤其是多核 CPU 的出现，任务逻辑的计算复杂性，计算的密集性，单线程的操作方式带来很大的不便，很少能满足当前的复杂 Web 网页的解析渲染，无法充分发挥计算机的计算能力，如图 25。

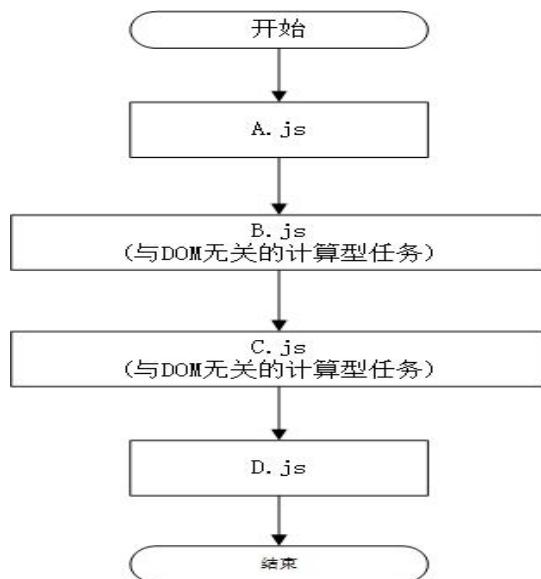


图 25 单线程运行模式

Figure 25 Single-threaded operation mode

在 ECMA 提出的多线程机制中，多个线程可同时运行在同一个进程中，和 C++、Java 一样的多线程机制，JavaScript 中的多线程可理解为多个独立 JavaScript 脚本，并且这些独立脚本由 JavaScript 引擎进行编译执行渲染，如图 26。

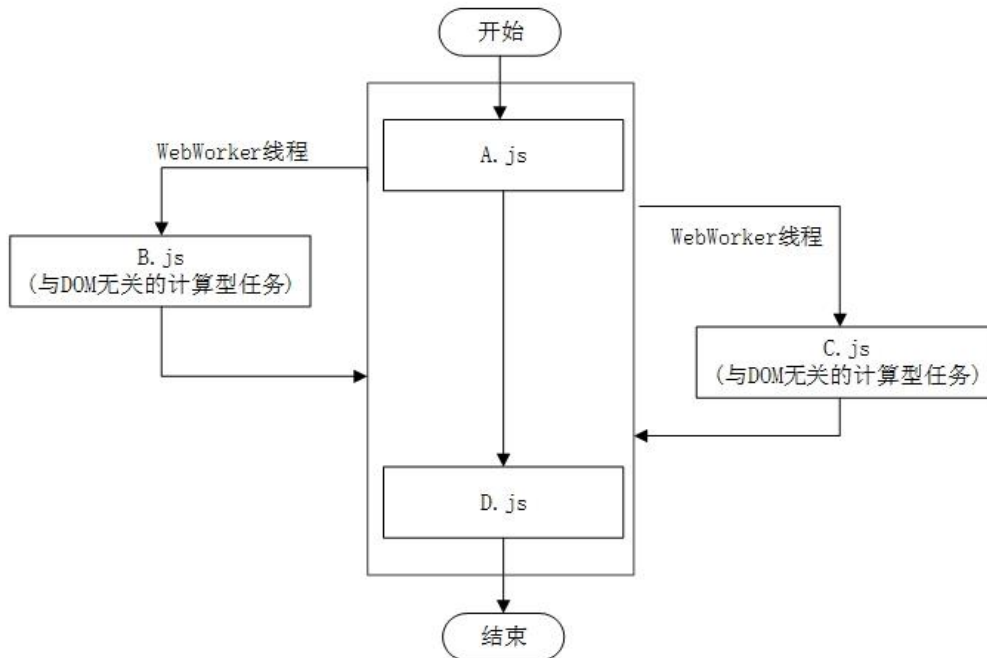


图 26 多线程运行模式

Figure 26 Multi-threaded operation mode

WebWorker 的基本原理是在 JavaScript 的主线程中，通过 `new Worker()` 创建独立的 Worker 对象用来开辟一个个相互独立的子线程，这些子线程互不阻塞，进行各自的逻辑计算过程，让本文的计算部分快速进行，尽量不影响展示效果。

独立的子线程是通过 `postMessage` 和 `onMessage` 两个接口和主线程进行交互，进行一些数据的交互通信等工作。

Web Worker 为 Web 内容在后台线程中运行脚本提供一种简单多线程共享内存运行方法。线程可以执行任务而不干扰用户界面。将一些任务分配给后者运行。在主线程运行的同时，Worker 线程在后台运行，两者互不干扰^[80]。但是 Worker 对象有自己的对象 `window`，和主线程的对象 `document` 不一样，以此导致子线程不含有 `dom` 对象，只能涉及到逻辑计算，不能涉及到 `dom` 操作。

因此本文的主线程主要用于一些简单的 `dom` 操作任务和一些异步操作人物，worker 线程主要进行各自的逻辑处理和计算，最后同时将计算结果交给 CPU 和浏览器引擎进行处理和渲染，通过这种多线程方式进行对本文的整体计算进行分解，最后达到原型系统的流畅运行和无卡顿交互操作，取得令人满意的可视化效果。

采用 Web Worker 多线程的并行计算方式来执行粒子系统的粒子生成、运动和灭亡等计算复杂性、密集性任务，对于本文的风场轨迹生成的过程涉及到粒子计算、差值计算、轨迹计算放在创建的多线程环境中进行计算，并将一些涉及到 DOM 操作的过程放在主线程中进行管理，并且主线程和 WebWorker 多线程之间通过 `postMessage` 和 `onMessage` 接口进行通信，实现过程如图 27。

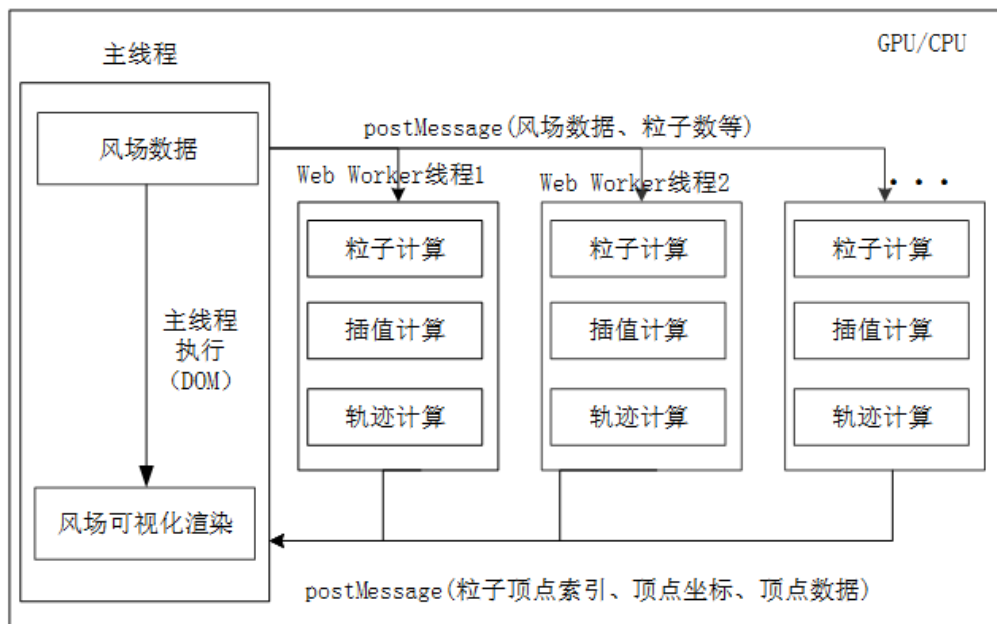


图 27 风场轨迹生成过程(逻辑计算过程)

Figure 27 Wind field trajectory generation process (logical calculation process)

4.1.1 实验结果和分析

本文验证 WebWorker 多线程的实验，是在相同粒子数量的情况下，通过设置 WebWorker 的不同数量进行测试，记录采用 Web Worker 的个数与计算时间的关系和不使用多线程计算时间的关系进而分析出 WebWorker 对本文设计的原型系统的风场数据的渲染效率的影响，本实验是在粒子数量为 40000 个的情况下得出的结果如表 1。

表 1 WebWorker 数量与计算时间的关系(单位: s)

Table 1 The relationship between the number of WebWorkers and the computing time

采用 WebWorker(个数)	多线程	不采用多线程
1	4.5	5.6
2	3.4	5.6
3	2.1	5.6
4	1.2	5.6
6	1.8	5.6
8	2.3	5.6

通过上表的结果得知, WebWorker 多线程的个数与计算时间的长短并不是成反比关系, 而是随着开启 WebWorker 的数量的不断增加, 整个原型系统的计算时间成先降后涨的规律, 呈现“U”形关系, 两者并不是成正比关系, 由此可知开启 WebWorker 的数量并不是越多越好, 要根据整个原型系统的渲染的数据量和计算量开启合适的 WebWorker 多线程个数, 这样才能最大限度保证本文所设计的原型系统得到流畅运行, 得到最佳的计算效率, 保证计算时间能尽量缩短, 避免页面出现页面“白屏”现象以及拖拽缩放时产生的卡顿现象, 尽量得到满意的结果。

4.2 基于多帧缓存区的加速算法

WebGL 是基于 OpenGL ES2.0 进行开发的, 是运行在现代 Web 浏览器上的 OpenGL ES 2.0 的 JavaScript 接口, 用于搭建动态二三位可视化 Web 应用的 API 接口, 因此用户可以通过 URL 进行远程访问用户所需要的 WebGL 二三维可视化可视化效果, 并且程序将运行与本地系统并利用本地的图形硬件 GPU, WebGL 的组织结构如图 28。

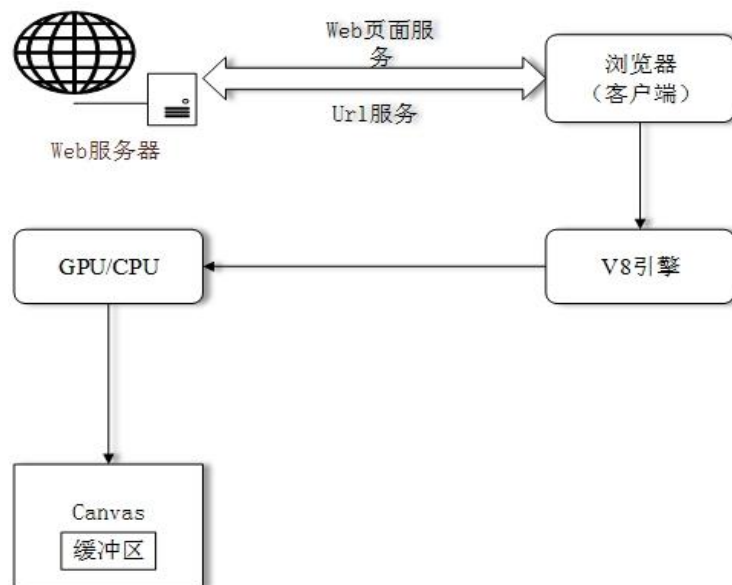


图 28 WebGL 的结构

Figure 28 Structure of WebGL

对于现代的 WebGL 应用程序，所有的绘制都是通过 GPU 的顶点着色器和片元着色器来控制的，利用帧缓存对象可以减少整个原型系统的数据交换，进而减少 CPU 的开销和内存的消耗。在 GPU 中开辟一个帧缓存对象来进行渲染操作，这种才做叫做离屏渲染(OffScreen Rendering)技术，WebGL 通过帧缓存对象提供离屏缓存操作的灵活性。帧缓存对象存在于图形内存中，因此可以拥有多个离屏缓存，这些离屏缓存之间的数据传输也不会使 CPU 和 GPU 之间的数据传输变慢，也不会增加本地系统开销。

本文利用创建多个帧缓存区内存的方法，将多帧缓冲区和多线程两者进行结合，将粒子渲染的相关属性存储到创建的帧缓冲区中，计算得到帧缓存区中的结果，最后将计算结果全部传输到主缓存区进行读写渲染，节省大部分时间，同时以上通过帧缓冲区对象渲染数据的过程全部在 GPU 中进行处理，大幅度减少 CPU 和 GPU 之间的数据交换和数据解析工作，尽量保证图形处理工作是在 GPU 中进行，CPU 仅进行最后的逻辑处理工作，这对整个系统的帧率有非常大的提升，具体的多帧缓冲区算法原理如图 29。

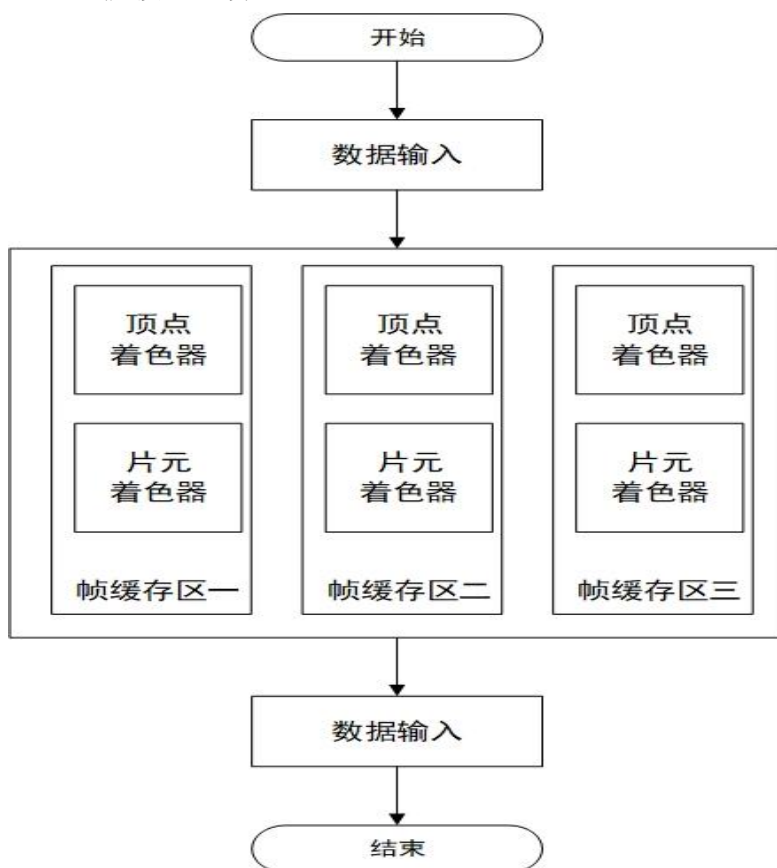


图 29 多帧缓冲区算法

Figure 29 Multi-Frame Buffer Algorithm

4.2.1 实验结果和分析

本文通过单独使用 GPU 和使用 GPU 结合 CPU 的两种不同的方式进行渲染不同个数的粒子数量，以此来验证本文采用的创建多个帧缓存对象的方法是否可行，以及在 GPU 中缓存所达到的效果，并记录风场数据的帧率，进而验证本文设计的原型系统的运行效率，得到的实验结果如表 2。

表 2 CPU 和 GPU 渲染效率对比表(单位: FPS)

Table 2 CPU and GPU rendering efficiency comparison table (unit: FPS)

粒子个数(万个)	GPU	CPU 和 GPU 结合
0.4	59	59
0.8	59	57
1	50	46
2	47	39
3	44	30
4	40	22
5	37	14
6.5	34	9

从表 2 中的实验结果数据可得知, 随着粒子个数的逐渐增加, 单独使用 GPU 渲染效率是远大于使用 GPU 结合 CPU 的渲染效率, 并且仅通过 GPU 的渲染效率是维持在 40 帧作用, 能够满足可视化场景的流畅运行, 不会出现卡顿现象, 由此可见本文所用的基于 GPU 的多帧缓存区对象的算法是具有可行性的, 能有效提高整个原型系统的运行效率, 进而达到整个原型系统的流畅运行。

根据本文提出多帧缓存区算法, 将涉及到的粒子渲染的过程放在 GPU 中进行计算, 只将最后的计算结果传送到 CPU 中进行最后的位置计算以及涉及到的逻辑计算, 可以有效解决本文提出的浏览器端 WebGL 的渲染效率低下, 帧率不符合要求的问题。

4.3 本章小结

本章从两个不同的方面介绍本文对全球风场可视化场景中的矢量场数据计算和渲染进行优化加速的方法, 即 WebWorker 多线程和基于 GPU 的多帧缓存区的加速算法。其中使用 WebWorker 多线程对本文中位置变化等逻辑计算进行抽取计算, 通过对比不同数量的 WebWorker 多线程, 找到最合适的多线程数量, 尽可能使本文中的逻辑计算效率得到最高, 这个方法对本文的原型系统的交互操作有较大的速度提升, 尽量避免页面出现加载慢, 白屏等现象。其次是针对本文中关于基于 GPU 的粒子渲染过程, 本文通过创建多个缓冲区对象的方式, 使得粒子渲染过程放在 GPU 中执行, 尽可能利用 GPU 强大的图形加速能力, 最后将计算结果一次性交给 CPU 进行抽取分离, 减少数据交换和数据解析工作, 对整个风场数据的动态渲染效率有较大的提升。

5 全球风场可视化原型系统设计与实现

在本章中，将本文的可视化方法和渲染加速方法应用到虚拟的“现实”场景中，通过计算机编码的形式，采用流行的 B/S 架构的软件设计理念进行软件设计，通过浏览器 Chrome 将所需求的场景表现出来。本文所设计的原型系统是基于三维 GIS 图形渲染引擎 Cesium 进行开发设计的风场可视化原型系统，并将本文设计的风场可视化原型系统与现有的相关产品进行对比分析。

5.1 基于 Cesium 的 B/S 架构系统构建方法

对于风场的可视化效果，要整体把握运行趋势和运行效果，本文对全球的风场进行可视化表达，但同时又要加上地理的元素，例如地形等元素，基于此设计理念，本文采用三维 GIS 渲染引擎 Cesium 进行搭建本文所设计的原型系统。

Cesium 是用于开发 B/S 架构的三维 GIS 应用程序的引擎，目的是为充分利用 3D GIS 数据的特点，尤其是 3D 地理数据结构，例如 3Dtiles、倾斜摄影等地理数据。Cesium 平台同时也提供开源和商用软件的结合的产品，是一套完成的工具，用于构建任何理性的 3D 地理空间应用程序，CesiumJS 是用于渲染二维地图、三维地球，并结合多种地理信息要素的 JavaScript 类库，Cesium 能够方面的切换二维、2.5 维和三维模式，并且能够对地理空间数据进行高精度和高性能并进行三维可视化。本文对 Cesium 的基本图元 Primitives 进行改写，重新改写 Cesium.DrawCommand 接口，对 DrawCommand 的底层 shader 进行重新定义，重新进行多管道渲染，并对代码采用 ES Module 的模块化进行代码组织，对本文的原型系统进行构建。

5.1.1 shader 自定义渲染

Cesium 是基于 WebGL 进行封装的三维 GIS 渲染引擎，是不需要任何浏览器插件，只需要浏览器支持 WebGL 功能即可。它封装 WebGL 底层复杂的功能函数实现，相关的 Cesium 开发者可无需接触复杂的 WebGL 原生函数和结构，不需要过于考虑顶点着色器、片元着色器以及纹理映射等问题。按照设计体系架构分，可将 Cesium 整个体系划分为 Core、Renderer、Scene 和 Dynamic Scene，如图 30。

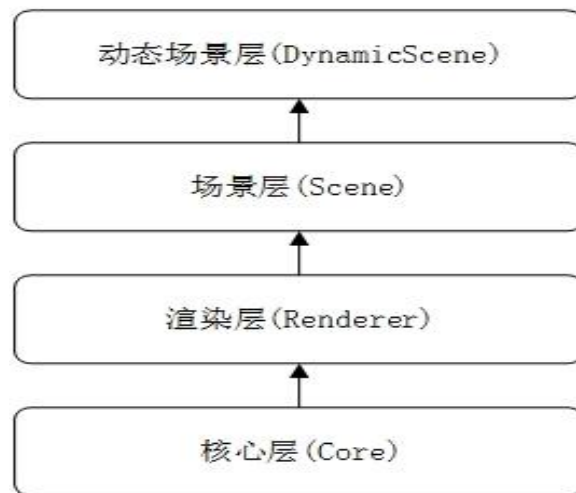


图 30 Cesium 体系架构

Figure 30 Cesium Architecture

封装渲染层的功能是可以让 GIS 开发人员不需要过深研究 WebGL 底层的渲染管线流程和操作底层相关功能函数，只需要操作渲染流程的 API 接口，更加方便操作，并且 Cesium 封装的相关的功能和格式更加符合 GIS 的要求。渲染层也内置 GLSL 常用的相关顶点、片元着色器以及相关渲染功能的封装接口，同时此层也包括着色器的顶点缓冲区、纹理以及渲染状态等结构，其渲染层的整体结构如图 31。

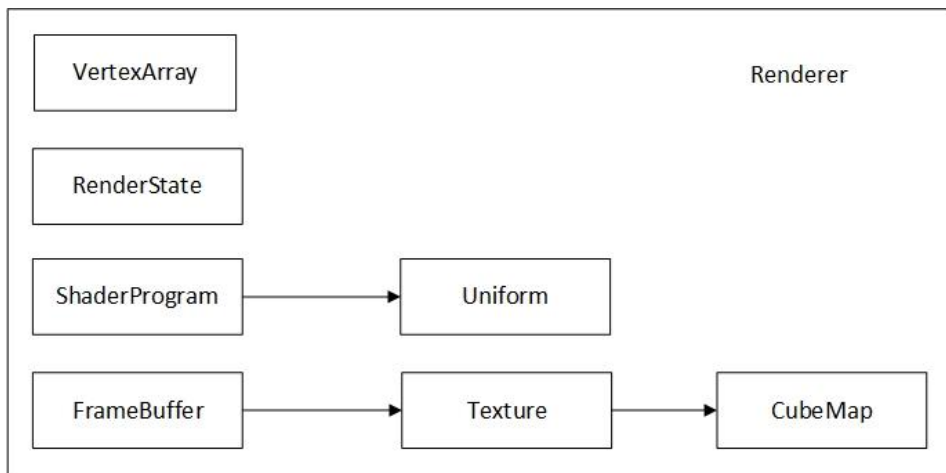


图 31 Cesium 渲染层结构

Figure 31 Cesium rendering layer structure

Cesium 中的绘制命令 DrawCommand 的基础是由上图中的 VertexArray、RenderState、ShaderProgram 和 Framebuffer 四个接口组成。

VertexArray: 是基于 WebGL 的顶点着色器 VertexShader 和顶点属性进行封装的接口,是具有一组顶点属性和可选索引的容器,属性和索引存储在缓冲区中。

RenderState: 包含发布绘制请求命令所需的状态，比如深度检测。

ShaderProgram: 是基于 WebGL 中的链接 Program 对象和 Uniforms 变量进行封装的 Cesium 接口，ShaderProgram 中有 vertexShader、fragmentShader 和 uniform 变量。这里的 Uniform 变量可直接使用 Cesium 中的笛卡尔坐标、纹理、矩阵和立方体地图等类型。

Framebuffer: 是封装 Cesium 中的纹理和渲染缓存的 Cesium 接口之一，也是 WebGL 的容器之一。

Cesium 在构建整个 JavaScript 库文件时提供一个大型的 GLSL 库，其中包含 function(函数)、uniform(Uniform 变量)、struct(结构)、constant(常量)。

5.1.2 自定义 Primitives 的复杂几何体渲染

Cesium 与 Ureal Engine^[81]、direct3d^[82]等三维图形渲染引擎类似，但 Cesium 的优势就在于大规模场景渲染，并且随着我们不断对 Cesium 的认知不断提高，就逐渐达到 Cesium 的目标领域：虚拟地球^[83]。Cesium.DrawCommand 是 Cesium 的绘制命令之一。在渲染层之上的是场景层，构建在场景层之上的是 Primitives(基本图元)，它们通过绘制命令来为提供场景层绘制虚拟的“真实”世界提供一些接口和绘制命令，如图 32。

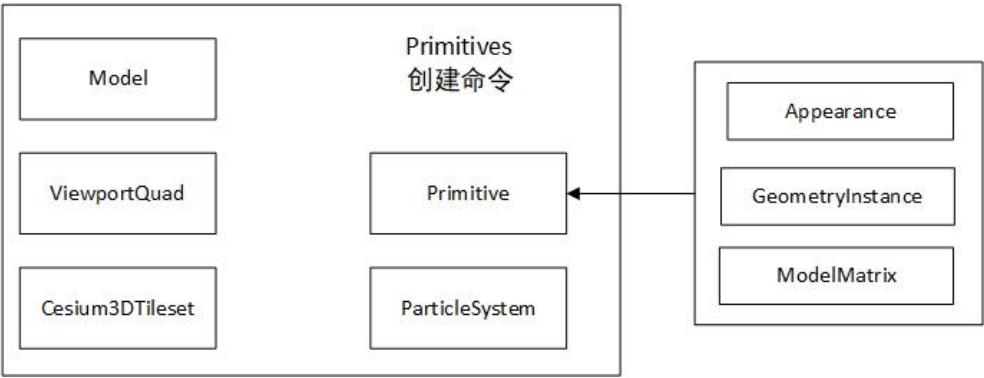


图 32 Primitives 结构图

Figure 32 Cesium Scene render layer

要执行自定义渲染，需要自定义 DrawCommand。要构建一个 DrawCommand，首先要逐个初始化它的组件，即 ShaderProgram、Texture、Uniforms 和 Framebuffer，然后将它们组合在一起创建一个 DrawCommand 对象。要将 DrawCommand 注入渲染引擎，需要一个自定义的原始对象，不需要实现 Cesium.Primitive 的所有方法，只需要 update、isDestroyed、destroy 方法即可。update 方法将在每次渲染开始之

前被调用。

5.2 基于 Cesium 的风场可视化原型系统实现

本文以 WebGIS 技术为技术支撑,解析风场数据,并根据微服务分层设计实现一个基于三维渲染引擎 Cesium 的风场数据可视化原型系统。该原型系统通过渲染数据粒子模拟全球风场的运动趋势,通过粒子属性表达本文所要求的风场效果展示,并针对本文的原型系统与现有的软件进行对比分析,突出本原型系统的自身的特点所在,展示效果良好、细节优化较好和地形因素等优点。

5.2.1 基于粒子系统的风场数据可视化原型系统构建

1)可视化系统运行环境

本原型系统是以 window10 操作系统进行开发的,同时 Cesium 是基于 Node^[84] 环境进行运行的引擎,因此本原型系统要基于 NodeJS 环境进行搭建,安装 node 版本为 14.17.4, NPM(Node Package Manage)版本为 6.14.14。为保证此原型系统能流畅运行,建议电脑配置 8GB 以上运行内存和 1660 独立显卡。三维粒子可视化需要依赖浏览器支持 WebGL,保证浏览器中展示效果和用户体验,因此要使用较新的浏览器,本文基于 Chrome 浏览器进行开发,同时适配 Microsoft Edge 浏览器。

2)原型系统数据库设计

MySQL

在本文中通过 MySQL 数据库来对本文研究所需要的数据进行存储和读写。MySQL 是一个关系型数据库,按照数据结构来组织、存储和管理数据的仓库,现隶属于 Oracle 公司,是基于 MySQL 企业版构建的完全托管的数据库服务,具有关系模型数据库的特点。

在本原型系统中,MySQL 用来存储风场数据。风场数据主要由每个格点的数据组成,其主要包含格网信息、经纬度坐标信息、风力和风向经纬度方向的 uv 分量。设计风场数据表结构如表 3。

表 3 风场数据表结构

Table 3 Structure of wind farm data table

字段名称	字段类型	字段说明
Id	Int	ID
lon	Float	经度
lat	Float	纬度
Lev	Int	格网等级
Force	Float	风力
U	Float	U 分量
V	Float	V 分量

根据上表建立相应的数据模型，并映射到 MySQL 中，并配置接口信息，供后端的数据渲染操作工作使用。

5.2.2 总体设计

1)原型系统模块结构

本文设计的可视化原型系统在设计过程中采用代理模式、异步模块设计模式和 MVVM 模式等多种设计模式，对不同功能的代码采用不同的设计模式进行封装，将不同功能模块封装在不同的 js 文件中，基于此设计理念，本文采用微服务架构体系，本文的原型系统层次体系主要包括网络服务层、基础数据服务层、逻辑处理层和场景展示层四个层次，如图 33。

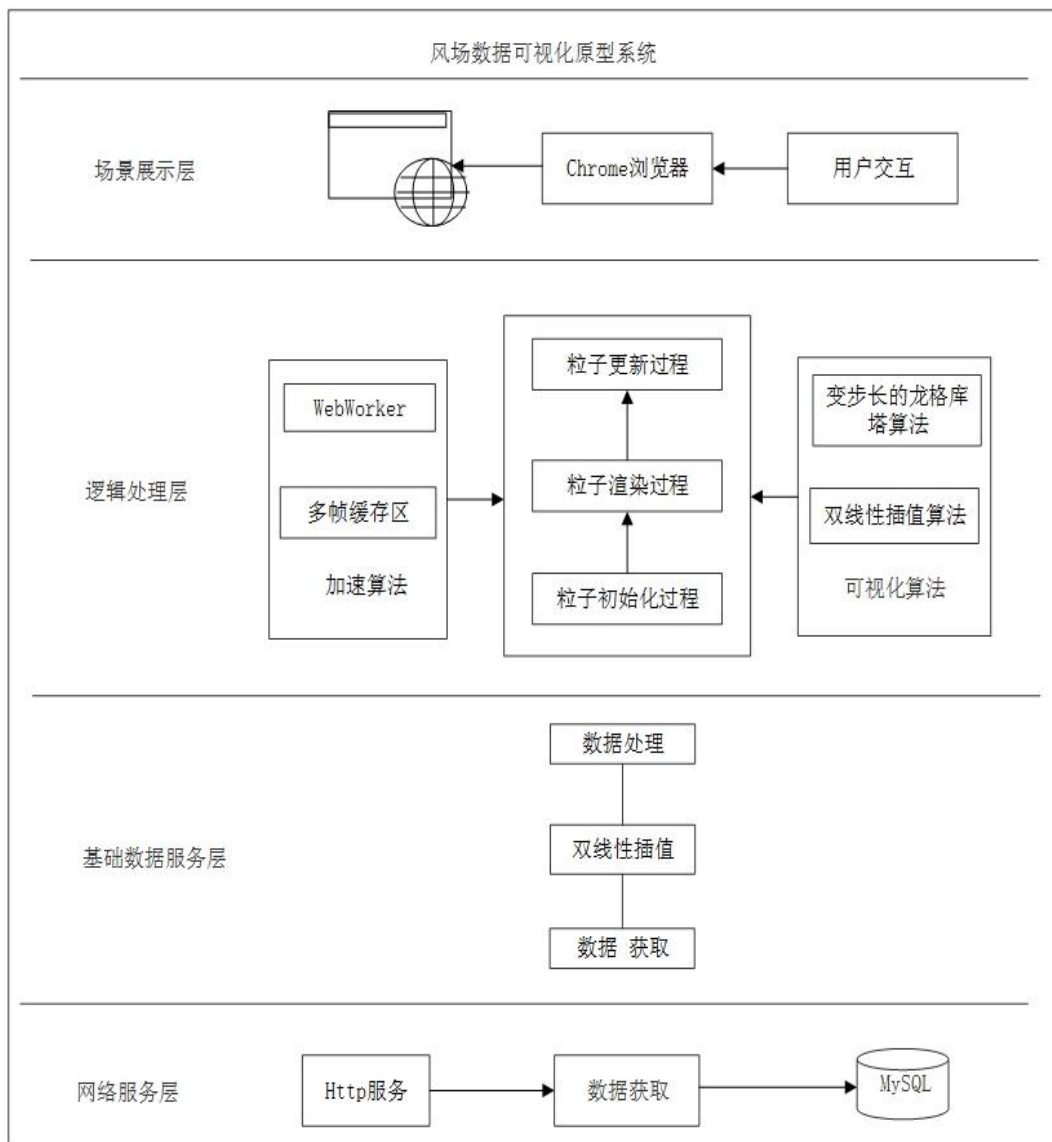


图 33 原型系统架构图

Figure 33 Schematic diagram of the prototype system architecture

从图 33 的原型系统架构图可以看出，各层之间的职能、联系和作用：

网络服务层：是原型系统的最底层，为整个原型系统提供网络服务以供其它层的使用，http、数据获取等网络服务等，从 NOAA 网站获取的数据进行结构化处理后存放到数据库中，以备接下来的操作使用；

基础数据服务层：第二层是数据服务层，为整个原型系统提供数据服务，主要是用来处理数据，主要是从数据库中读取数据，对数据采用双线性插值算法进行处理，使得数据的分辨率尽量符合要求，以满足下一步的要求；

逻辑处理层：第三层是逻辑处理层，是整个原型系统的核心层，主要是进行算法的实现，主要实现本文的粒子可视化算法，实现本文的基于矢量场双线性插

值算法和基于 GPU 的变步长的四阶龙格库塔算法，主要为粒子渲染 t 提供一定的理论基础和逻辑计算服务；

场景展示层：最顶层，负责向用户显示和解释用户指令，主要用来展示风场数据的可视化界面，界面包括粒子运动轨迹模块，DOM 影像模块，用户交互功能模块，其可以控制粒子的生命周期、运动速率、粒子的大小、形状等功能。

这种微服务化的层次结构是基于各个微服务所独有的功能，每一层结构业务单一简单，可以被轻易开发出来，使得研究人员专注于某种单一的服务。基于它们所提供的服务，可以被独立地部署到应用中，微服务可以看作相互独立的组件，可根据需求升级或替换某一组件，不需要修改全部组件，尽量降低模块之间的耦合度，提高开发速度。

2) 技术结构

本文设计的原型系统是遵循前后端分离原则，PC 端和服务端主要遵循 http 协议进行设计接口实现数据交互，减少开发的难度，提高开发效率。服务端框架采用较为流行的 koa 框架，koa 是基于 Node 服务器和 JavaScript 语言进行设计的业务逻辑服务服务器框架。PC 端的展示主要通过三维图形渲染引擎 Cesium 和前端框架 Vue 进行场景的设计，Vue 主要服务构建前端功能模块化并结合 ESM module 为本文的微服务体系提供一定的支持，以便于能够独立开发相关功能模块，Cesium 是负责风场展示、粒子绘制等操作。借助 Cesium 来渲染粒子可视化效果有助于提高开发效率，无需大规模改写相关内容。基于此实现想法，提出的技术框架，如图 34。

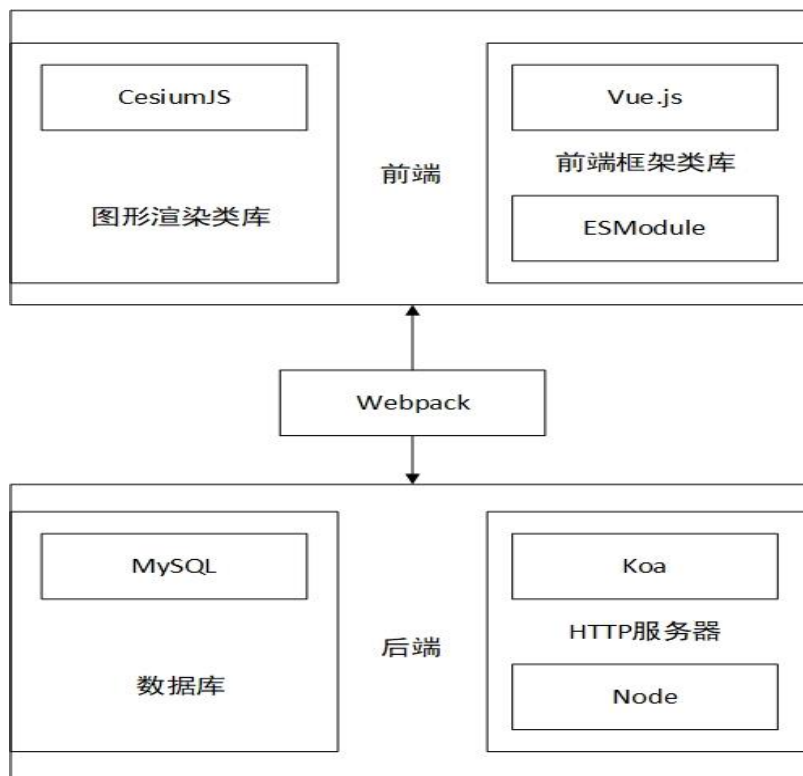


图 34 系统技术结构图

Figure 34 System technical structure diagram

本原型系统的后端部分是基于 Nodejs 环境搭建 HTTP 服务，借助 koa 服务框架和 nginx 服务器设置反向代理解决跨域问题，设置网络服务获取数据。通过所设置的跨域代理请求向 NOAA 请求获取所需要的风场数据，然后将所得到的数据进行结构化处理存储到数据库中，本文的数据库采用 MySQL 来进行存储和读写风场的等结构化数据。Koa 解决异步流程控制和异常捕获两大问题，使得框架变得轻便，Nginx 服务器提供的反向代理，减少跨域的成本，可以使前端开发人员不为跨域问题浪费时间，很大程度上提高开发效率。本原型系统主要通过 koa 和 nginx 构建提供基于 https 协议的数据访问接口。配置好 MySQL 连接组件，就可以连接到本文配置好的 MySQL 数据库中，进行数据的读写和存储。

PC 端部分主要采用三维图形渲染引擎 Cesium 以及 JavaScript 代码框架类库。对于所涉及到的正射影像 DOM、地形地图等基础数据通过发布服务的形式加载，所需要的矢量场数据服务通过三维 GIS 渲染引擎 Cesium 进行渲染。通过改写 Cesium 的相关 WebGL 函数并调用相关 WebGL 的 API 接口进行风场数据处理操作，改写相关的函数和接口，改善渲染效果，提高开发效率，尽可能的节约开发成本。对本文的 JavaScript 代码通过 ES Module 的代码组织方式进行模块化组织，其中 ES Module 都是 JavaScript 文件和模块加载器，是前端代码规范标准。ES Module

针对浏览器进行优化，并且可在 Node 环境中直接进行使用。

对于本文的原型系统的工程搭建通过 webpack 进行搭建，对所编写的代码进行打包。本质上，webpack 是一个现代 JavaScript 应用程序的静态模块打包器。当 webpack 处理应用程序时，它会递归地构建一个依赖关系图(dependency graph)，其中包含应用程序需要的每个模块，然后将所有这些模块打包成一个或者多个 bundle。本文的代码文件相互依赖，不方便进行管理，从这些文件的入口起始文件开始，webpack 会递归构建一个依赖图，这个依赖图包含应用程序所需的每个模块，然后将所有这些模块打包为少量的 bundle，可由浏览器加载。

3)系统主要流程

本原型系统是通过粒子模拟风场的可视化效果。其中风场数据是从 NOAA 官方网站通过代理设置获取合适的大气层的风场数据，本文设计的原型系统需要对获取的数据进行处理，对风场数据进行格式转化，配置数据模型并设置字段存入到 MySQL 数据库，供数据渲染操作使用。

本文的总体处理流程分为两部分，第一部分是数据处理流程，主要是获取风场数据，将数据经过结构化处理后存储到数据库中；第二部分是风场数据动态可视化流程，将数据通过粒子的形式渲染展现出来，用户也可自行控制粒子的形状、大小、速率、数量等因素。

根据设计思想，其中数据处理的主要流程如图 35

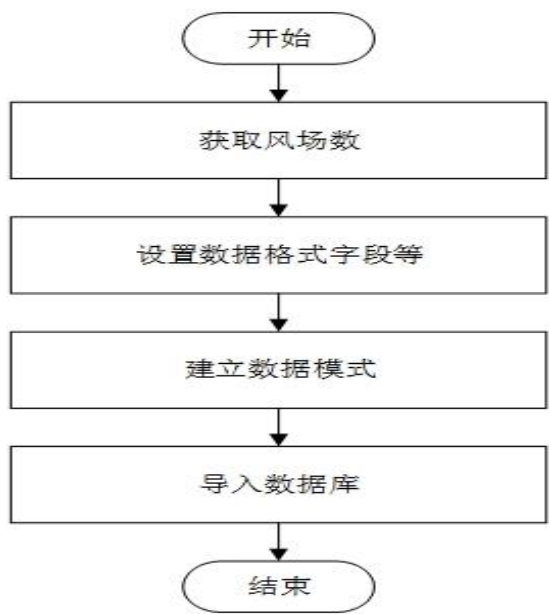


图 35 风场数据处理流程

Figure 35 Wind farm data processing flow

其中 Web 端场景展示主要流程如图 36:

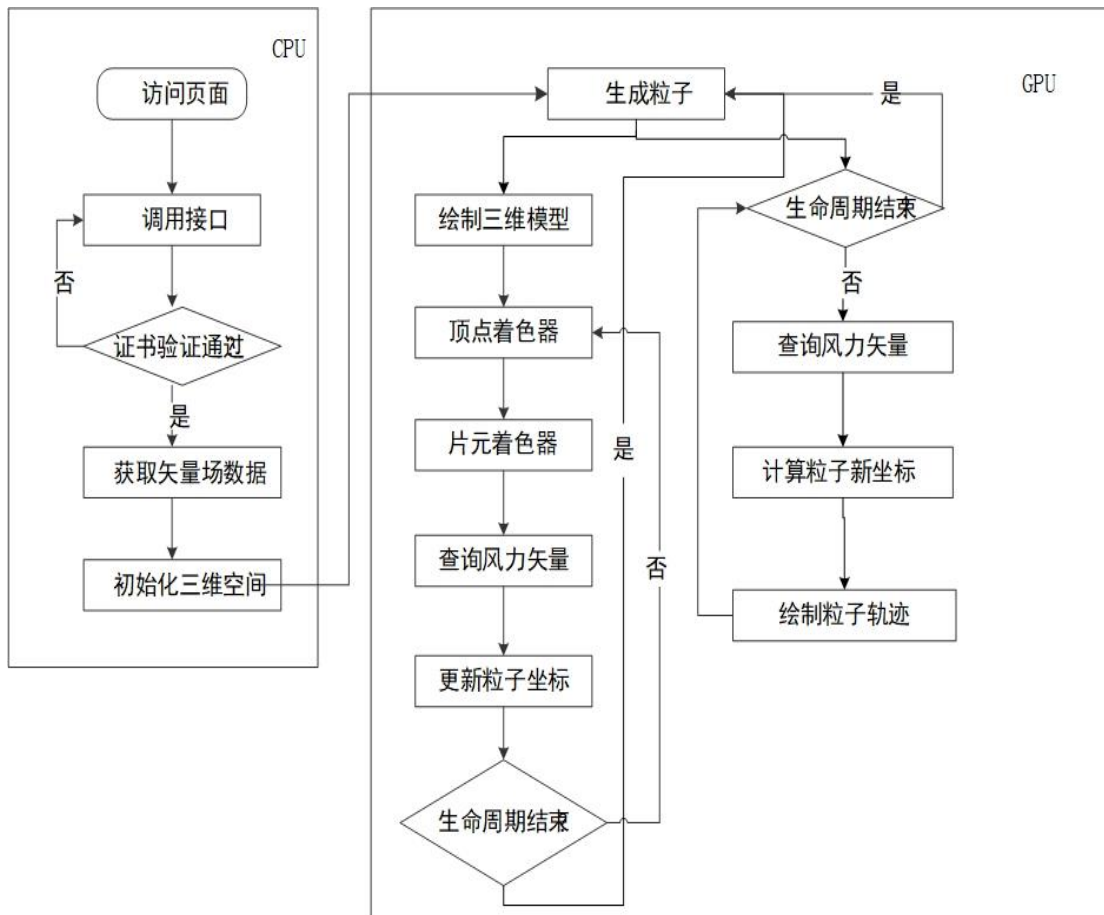


图 36 web 场景展示流程

Figure 36 Web scene display process

5.2.3 可视化系统模块实现

1) 界面设计

本原型系统的界面比较简洁，仅有标题栏、用户交互控制栏和场景展示模块，如图 37。

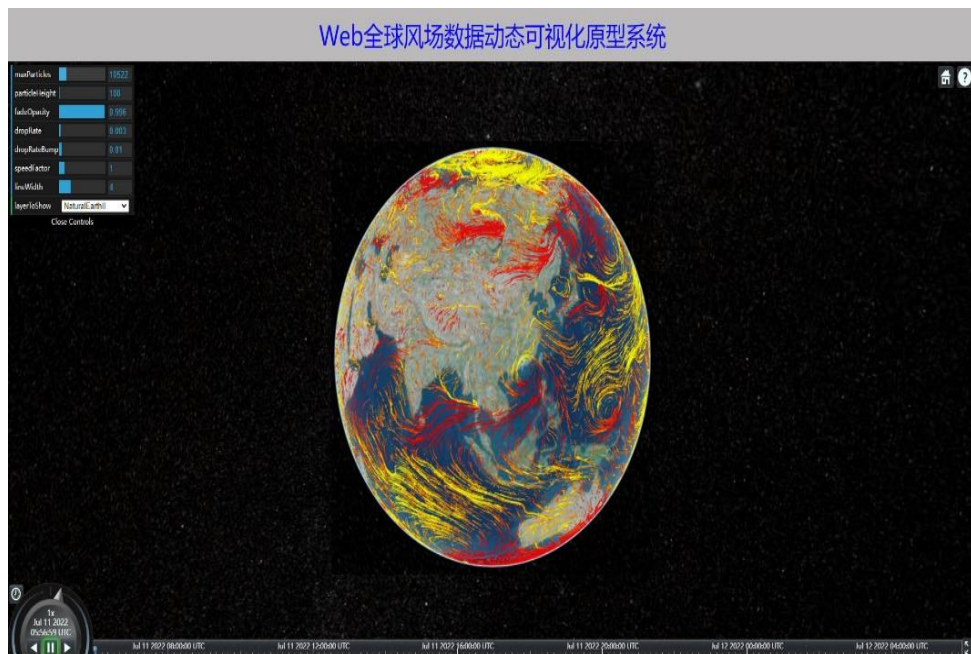


图 37 初始化界面

Figure 37 Initialization interface

2) GUI 控制

GUI(Graphical User Interface, 简称 GUI, 中文名是图形用户界面), 是指通过图形的方式显示计算机操作用户的界面, 允许用户通过键盘、鼠标或者手柄等输入设备操作屏幕上的图标或者菜单选项, 以选择命令、调用文件、启动程序或执行其它一些日常任务^[86]。本文采用 `dat.gui` 进行用户控制界面的设计, `dat.gui` 是用于在 JavaScript 中更改变量的轻量级图形用户界面。这些构建的 JavaScript 文件捆绑运行 `dat.gui` 所需的所有依赖项。本文设置好的 GUI 界面, 如图 38。用户可以通过控制 GUI 面板调控本文设置的一些关于粒子系统的粒子数量、速率、大小等属性。

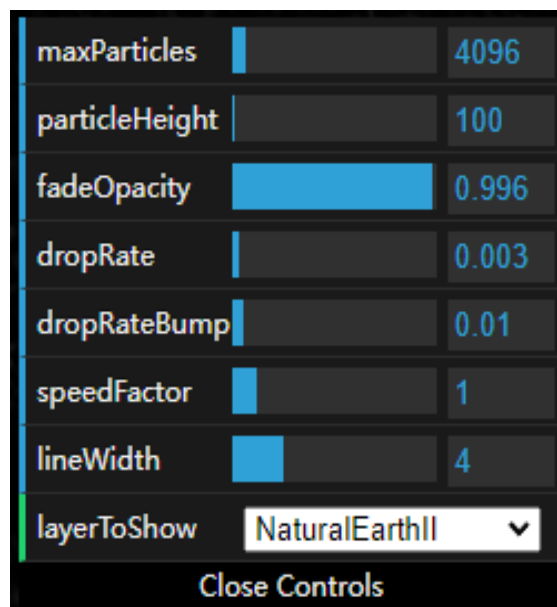


图 38 初始化 GUI 界面

Figure 38 Initialization GUI interface

3) 三维风场数据动态可视化渲染模块

在打开浏览器的同时，浏览器会根据相对应的 JavaScript 文件进行编译执行，浏览器会根据本文设置的接口进行请求数据，通过代理设置，验证通过后，会获取到 nc 数据，然后对数据进行解析，抽取所需要的风场数据，最后将所获的风场数据经过结构化处理转换成符合要求的格式后更新到数据库中。

接下来是处理粒子可视化展示，先根据数据量的大小和浏览器的尺寸生成相对应的粒子数量，根据风场数据的点分布情况和数量设置粒子的生命周期，然后根据 NC 文件的 UV 分量、角度等值跟踪设置下一个粒子的位置坐标。同时根据数据的密集、风力大小情况和粒子的变化赋予粒子颜色不同的深浅，绘制粒子的运动轨迹，并根据风场数据和粒子的特点随时更新和调整粒子生命周期的状态。

同时为保证粒子的运动效果和效率，浏览器需要快速并有效地刷新画布信息，帧率(FPS)就是一个重要的评判标准，每次刷新都需要重新更新粒子的状态，更新运动轨迹和更新粒子生命周期。利用 GPU 的计算图形处理的优势，绘制粒子运动轨迹的同时提高运算速度，提高粒子的渲染效率，同时利用 WebWorker 多线程进行并行计算，提高计算速度，对原型系统的整体运行效率进行提高，提高运算效率。通过 GPU 和 WebWorker 两者的加速运行，会尽量保证帧率满足要求，到达 40 帧左右。在同一时刻要准确判断粒子的生命周期是否处于“消亡”状态，粒子生命周期若处于此状态代表粒子消失，会重新生成粒子，进行新一次的生命周期活动，没有处于此状态的粒子会继续活动直到粒子消失。

基于以上实现思路，本文实现的三维粒子数据渲染模块的实现流程，如图 39。

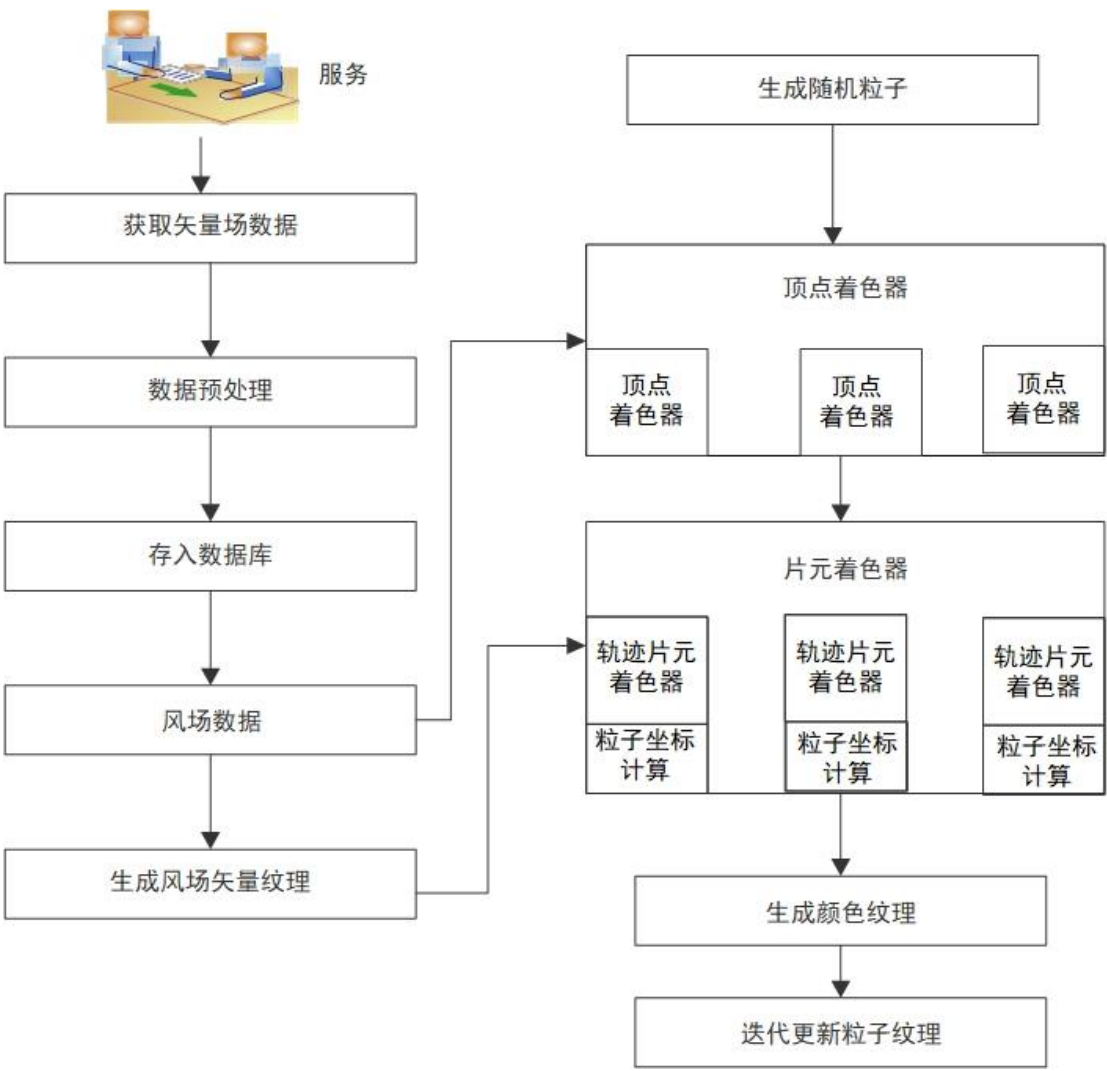


图 39 三维粒子风场数据可视化流程

Figure 39 Data visualization process of 3D particle wind field

Ceism 的 shaderProgram 接口基于 WebGL 的管道渲染技术进行设计，可通过采用原始的 shader 语言编写粒子的可视化代码，实现粒子的片元属性绘制，例如粒子形状、粒子颜色、数量、坐标等属性可视化。由于 GPU 的图形处理的优势和 shader 在各阶段的优化处理，采用 GPU 和 WebWorker 两者结合的方式渲染粒子可以有效的减少浏览器的消耗，尽量提高浏览器运行代码的效率，优化用户体验。

本文关于粒子渲染的顶点着色器相关代码如附录 1。顶点着色器的主要作用是用来初始化粒子，根据相关的纹理信息绘制粒子，在初始化之前会判断粒子处

于生命周期的所处的阶段，并自行进行销毁生成等操作。伴随着顶点着色器完成之后开始执行片元着色器。

本文的片元着色器分为粒子速度片元、粒子轨迹片元着色器两种片元着色器，粒子速度片元着色器是负责粒子速度的更新，粒子速度片元着色器的相关代码如算法附录 2，并结合粒子轨迹片元着色器进行对粒子进行跟踪绘制粒子轨迹，从而绘制出风场的轨迹。粒子轨迹片元着色器算法如附录 3，并将最后的结果生成纹理信息交给顶点着色器处理，并不断更新粒子。

5.2.4 可视化系统结果与测试

本文设计的原型系统是借助粒子运动模拟的方式来模拟风流动情况，以风场数据作为基础进行可视化展示。硬件环境为处理器 Inter Core i5 2.7GHz；内存为 8GB；硬盘大小为 500GB；显卡为 Intel GTX1660。软件环境为 Google Chrome 57.0 浏览器。本文选取全球风场数据进行可视化，并以多角度、多影像对可视化效果进行查看，本原型系统主要包含三维空间下的基于粒子系统的风场可视化。

本文的原型系统是基于强化功能、简化界面的设计原则设计。图 40 中的线条是代表粒子的运动轨迹，线条的绘制方向是代表粒子运动的方向，即可代表当前数据点的风力方向；粒子的生命周期是通过粒子运动的距离进行体现的，因此显示的流线长度代表粒子生命周期的长短，也就是风力矢量的距离。从图 40 可以看出整体的渲染效果，较为清晰的表达出风场在全球的分布情况。

图 40 中的粒子整体渲染的效果，非常清晰的看出整体的风场分布情况和运动趋势，也很方便的看出风力中心点的位置。同时将风场数据映射为不同的颜色在粒子运动是通过展现出来，通过不同的颜色可以看出风速的不同。总体来看，三维粒子可视化展示效果更加直观、立体感强。

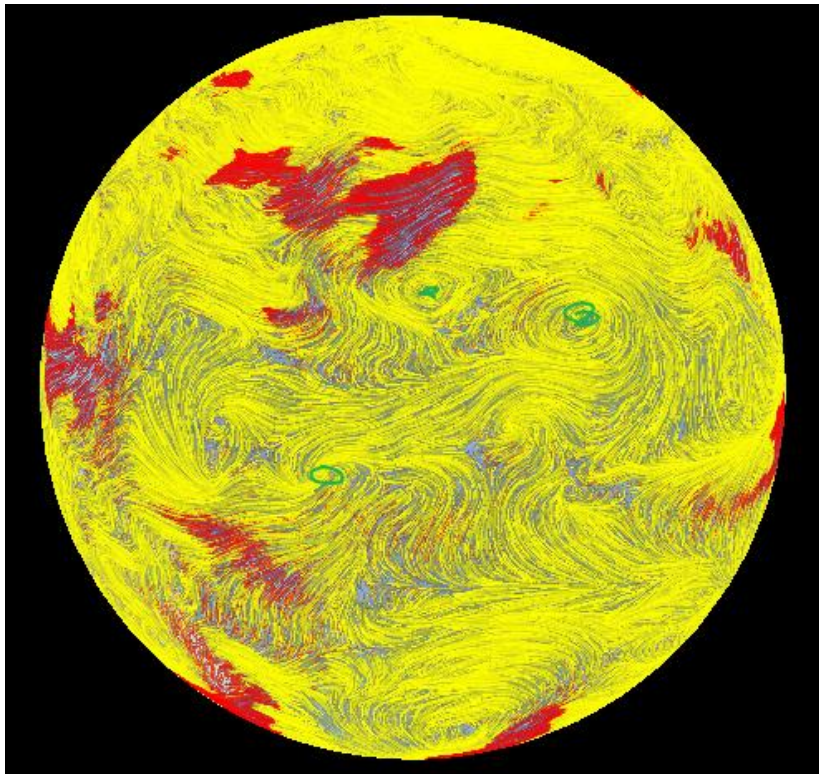


图 40 三维粒子可视化展示

Figure 40 3D particle visualization display

5.3 系统对比分析

本文设计的全球风场动态可视化原型系统具有一定的实用性和创新性，并与现有的相关天气数据可视化产品 NullSchool 全球天气数据可视化系统^[85]进行对比。

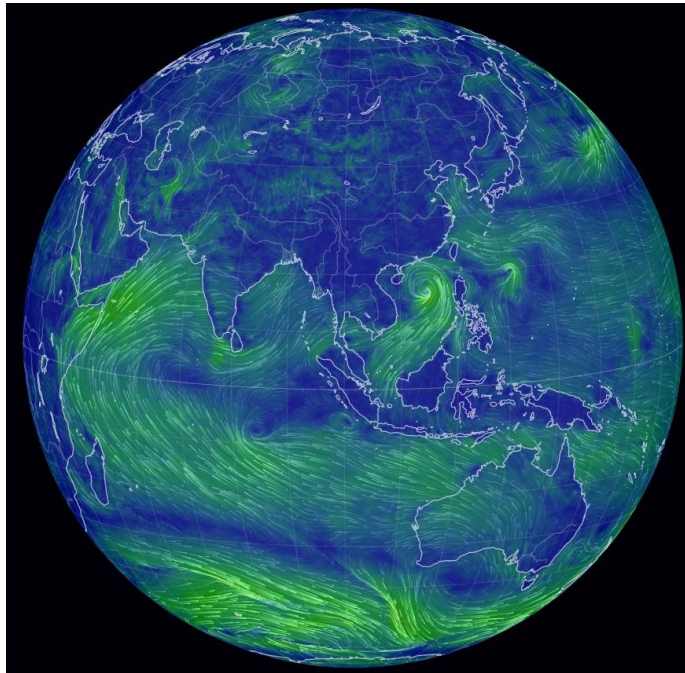


图 41 NullSchool 天气数据可视化^[85]

Figure 41 NullSchool weather data visualization

图 41 为 NullSchool 的全球天气数据可视化产品，绘制效果较好，但角度单一，不能多角度观察风场变化，且没有地形，仅能从整体趋势进行分析，仅限于从效果的角度出发，不符合 GIS 的方向。

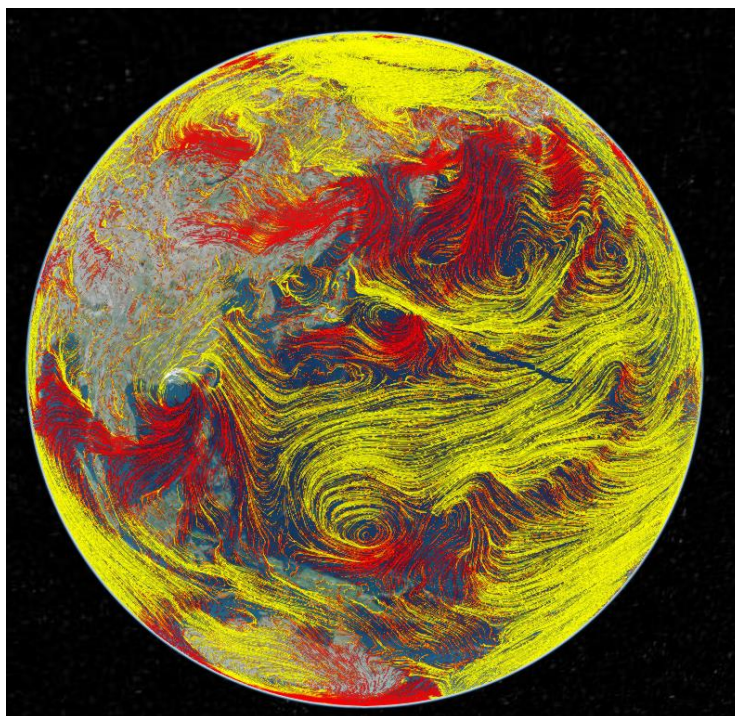


图 42 本文所设计的风场数据可视化

Figure 42 The wind farm data visualization designed in this paper

图 42 为本文设计的风场原型可视化原型系统，是基于粒子系统的可视化效果，粒子运动轨迹的绘制和颜色的绘制是同时进行的，粒子展示效果好，轨迹绘制效果较好，并且本原型系统可以多角度查看风场的渲染效果，并且也有地形的效果，还可以研究地形对风场的影响，如图 43。

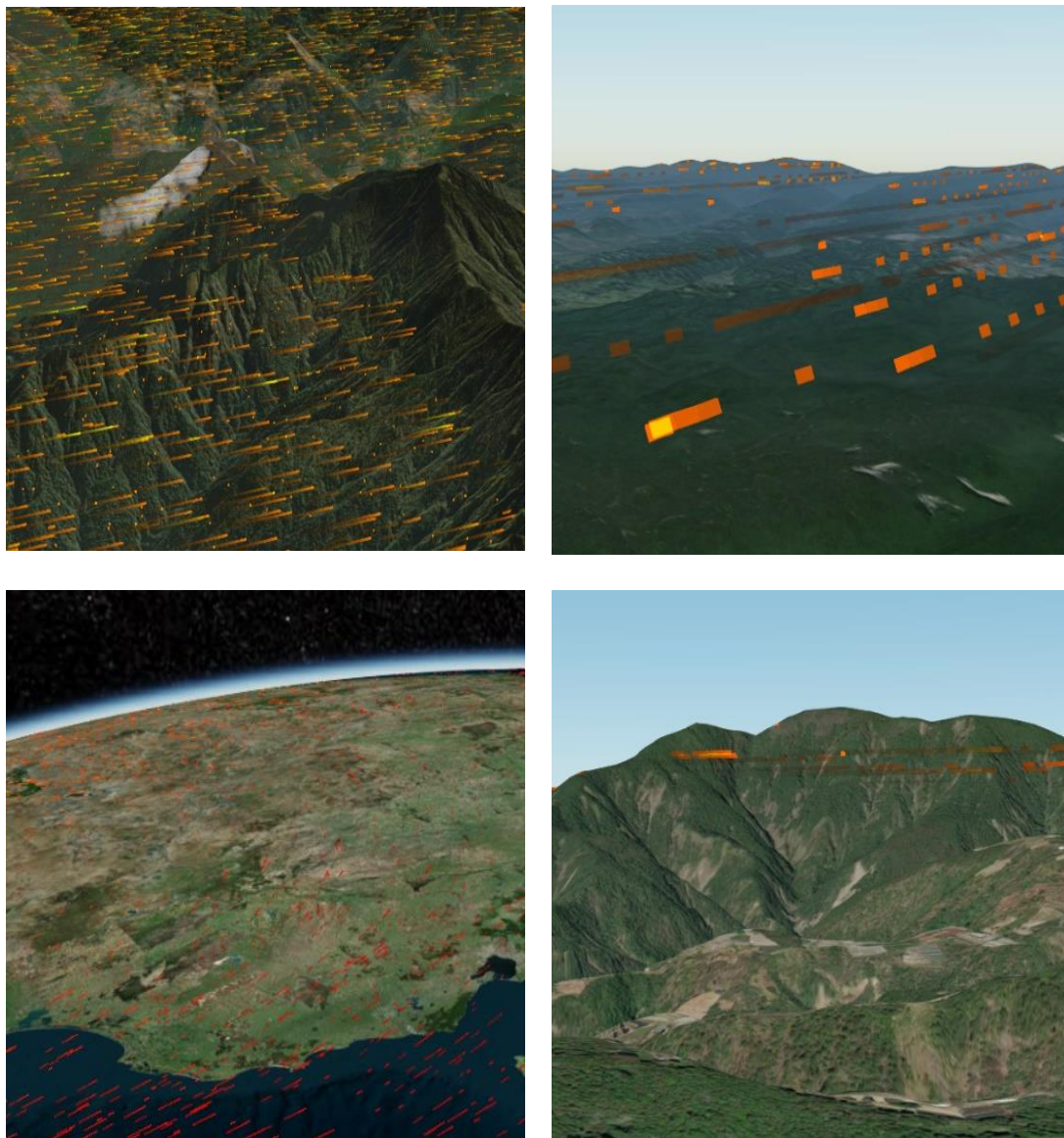


图 43 地形影像的风场可视化（多角度）

Figure 43 Visualization of wind field with topographic imagery

综上所述，本文从地形、多角度观察、符合 GIS 方向和绘制速度和效果等多个评价因子进行对比分析，得到结果如表 4。

表 4 可视化系统对比分析结果

Table 4 Visualize the results of the system comparison

	NullSchool	原型系统
是否有地形	否	是
是否可多角度观察	否	是
是否符合 GIS 方向	否	是
粒子绘制速度	快	快
绘制效果	良好	良好

从上表可以看出本文设计的原型系统在地形、多角度观察、是否符合 GIS 方向以及绘制速度和效果等方面较 NullSchool 有一定的优势，可说明本文设计的原型系统具有一定的独特性和优势，说明本文所采用的可视化算法和加速算法是具有一定优势和创新。

5.4 本章小结

本章主要从基于粒子系统的风场数据动态可视化原型系统设计与实现和基于 Cesium 引擎搭建全球 Web 风场可视化原型系统两方面进行研究和论述。第一部分是基于 Cesium 构建的全球 Web 风场可视化原型系统，分别对自定义 shader 和自定义 Primitives 复杂几何体进行构造，同时对渲染模块通过 ESMdule 组织代码块。第二部分是对风场的动态可视化进行科学和详细的设计，采用现如今比较流行的微前端架构，并通过编码的方式实现，最后，对实现结果进行展示，并和其它现有产品进行简单的分析对比，从可视化效果和 GIS 的角度等多方面进行对比分析，可知本文改进的四阶龙格库塔算法和设计的全球风场动态可视化原型系统具有一定的独特性和创新。

6 结论与展望

6.1 结论

在地理学领域中，通过对矢量场数据的可视化渲染，可以直观的展现矢量场数据。因此矢量场可视化是解释矢量场运动变化规律的有力工具，在工程分析和科学计算中发挥着十分重要的作用，具有非常重要的研究价值和现实意义。本文基于 GIS 开源平台 Cesium，通过研究国内外先进的矢量场数据动态可视化的相关方法和技术，结合可视化软件和硬件，利用龙格库塔算法绘制实现全球风场表达，同时针对风场数据的分布特点，对龙格库塔算法进行改进，并对其进行验证，最后将本文研究的算法应用到本文所设计的原型系统中。本文的主要工作和成果如下：

分析并研究风场数据的获取、解析和可视化流程，针对矢量场数据进行提出和筛选提取出符合要求的数据。

对粒子的生成算法进行研究，先利用双线性插值算法对数据进行处理，提高风场数据的分辨率，提高粒子初始化分布的均匀性，并利用四阶龙格库塔算法对粒子绘制轨迹，并对粒子轨迹情况通过改变步长的方式进行优化，优化风场可视化的表达，改善风场数据的渲染效果。本文通过实验验证本文提出来的基于 GPU 的变步长的龙格库塔算法是具有可行性的。

利用 WebWorker 多线程的方式，对本文的非 DOM 操作进行 WebWorker 多线程的操作，创建合理 WebWorker 的数量，可以最大限度的提高并行计算能力，同时利用创建多个缓存区对象的方法，尽可能利用 GPU 的图形处理的能力，通过这两者的加速方法，加快原型系统的可视化处理速度。并通过实验验证本文的加速方法，得知有效地提高可视化原型系统的计算加速能力和渲染效率。

依托三维 GIS 开源平台 Cesium 进行设计和开发基于粒子系统的风场数据可视化原型系统。设计和实现风场数据动态可视化模块，该模块主要包括数据处理模块、动态可视化渲染模块、参数配置模块。其中服务端数据处理是对相关的风场数据进行下载调用，并对调用的数据进行预处理并入库；动态可视化模块对数据进行渲染，进行可视化展示；参数配置模块对相关的粒子属性进行配置和调节，封装本文的三大模块并集成设计的原型系统，并与其它成型的相关产品进行对比分析，验证本文的原型系统的独特性和实用性。

6.2 展望

因为对相关技术的研究时间过短，本文对风场的矢量场数据的动态可视化的研究深度和广度都存在一定的局限性，解的不够透彻，因此本文的可视化研究方法还是存在一些缺点的，存在较多的局限性，因此未来的工作要从以下几个方面进行深一步的研究：

1) 基于粒子系统的可视化效果改进

在基于粒子系统的可视化基础上，通过改进四阶龙格库塔算法对风场效果进行动态可视化的展示效果有所改进，显示风场的动态变化趋势，但对于风场的动态变化规律还缺乏有力的验证。因此要在下一步的矢量场可视化过程中，要更多的考虑风场数据的内在联系和特性，在保证可视化能满足要求的前提下，要更直观地展示可视化效果和揭示数据间的联系。

2) 全球场景可视化渲染效率的提高

针对本文的全球场景可视化表达出现的帧率不能满足要求的情况，本文分别从提高 CPU 的计算能力以及利用 GPU 的图形处理能力两方面出发进行效率方面的渲染，取得良好的效果。但因数据量过多，当发生在配置低的计算机上，内存消耗极为明显。因此在接下来的工作中，要结合多种合适的渲染加速算法，合理分配计算机资源，找出每种方法的最佳的运行时间，提高计算能力，充分利用并合理使用 CPU 和 GPU 的资源，进一步提高数据的渲染效率。

3) 大气矢量场数据联合

大气环境是一个复杂的环境，大气数据不仅包括风，还包括各种污染物、盐度、湿度等矢量场数据。单一矢量场数据难以揭示和反应复杂的大气环境，并且单一数据仅表现自己的表现趋势和规律，无法联动其他数据，反映对其他数据的影响。在可视化过程中，数据之间的关系并不是单独存在的，一个要素的改变肯定会影响到周围环境和要素的改变，不同源数据之间会互相影响。因此本文以后的研究方向，要尽量多种大气数据结合集成进行可视化表达和分析。

参考文献

- [1] 郭华东.空间对地观测与新一代数字地球[J].中国建设信息,2012(21):21-22.
- [2] 安培浚,刘细文,李佳蕾,华旦草.趋势观察: 国际地球大数据领域研究态势与热点趋势[J].中国科学院院刊,2021,36(08):989-992.
- [3] <https://www.earthobservations.org/geoss.php>
- [4] D. H. Laidlaw et al., "Quantitative comparative evaluation of 2D vector field visualization methods," Proceedings Visualization, 2001. VIS '01., 2001, pp. 143-150, doi: 10.1109/VISUAL.2001.964505.
- [5] 单扬洋. 气象矢量场数据可视化技术研究[D].浙江工业大学,2019.
- [6] 黄晶慧.颜色与点图标方法的可视化[J].电脑知识与技术,2015,11(01):153-156.
- [7] 宋丽丽. 面向海量的海洋数据动态可视化与实时绘制技术研究[D].中国海洋大学,2013.
- [8] 郭晓非,朱俊利,万剑华,汪小琪.基于 Cesium 的海洋环境要素三维可视化研究[J].海洋科学,2021,45(05):130-136.
- [9] 宋伟杰,崔俊芝,叶正麟.基于面图标的三维对称张量场可视化研究[J].工程图学学报,2010,31(06):146-150.
- [10] 樊宇,吕晓琪,张继凯,王月明,张信雪.海洋风场动态可视化研究[J].应用海洋学学报,2018,37(02):173-178.
- [11] 王想红. 基于三维虚拟地球的海洋环境数据动态可视化研究[D].辽宁工程技术大学,2013.
- [12] N. Max, R. Crawfis and C. Grant, "Visualizing 3D velocity fields near contour surfaces," Proceedings Visualization '94, 1994, pp. 248-255, doi: 10.1109/VISUAL.1994.346312.
- [13] Y. Chen, J. Cohen and J. Krolík, "Similarity-Guided Streamline Placement with Error Evaluation," in IEEE Transactions on Visualization and Computer Graphics, vol. 13, no. 6, pp. 1448-1455, Nov.-Dec. 2007.
- [14] L. Li and H. Shen, "Image-based streamline generation and rendering," in IEEE Transactions on Visualization and Computer Graphics, vol. 13, no. 3, pp. 630-640, May-June 2007.
- [15] M. Zockler, D. Stalling and H. . -C. Hege, "Interactive visualization of 3D-vector fields using illuminated stream lines," Proceedings of Seventh Annual IEEE Visualization '96, 1996, pp. 107-113.
- [16] 彭宝云,王文珂,李思昆.大规模流场矢量线可视化的数据预取方法[J].计算机辅助设计与图形学学报,2016,28(03):464-470.
- [17] D. N. Kenwright and G. D. Mallinson, "A 3-D streamline tracking algorithm using dual stream functions," Proceedings Visualization '92, 1992, pp. 62-68.

- [18] Feng Y , Shi E , Luo Y , et al. Implementation of streamline simulation based on finite element method in FEniCS[J]. Computational Geosciences, 2020, 24(1):333-347.
- [19] Ji P, Tian F, Liu S, et al. 3D streamline visualization for irregular flow field data[J]. Journal of Computational and Theoretical Nanoscience, 2016, 13(11): 8909-8916.
- [20] C. -K. Yeh, Z. Liu, I. -H. Lin, E. Zhang and T. -Y. Lee, "WYSIWYG Design of Hypnotic Line Art" in IEEE Transactions on Visualization and Computer Graphics, vol. 28, no. 6, pp. 2517-2529, 1 June 2022, doi: 10.1109/TVCG.2020.3032734.
- [21] A. S. Ali, A. S. Hussien, M. F. Tolba and A. H. Youssef, "Visualization of large time-varying vector data" 2010 3rd International Conference on Computer Science and Information Technology, 2010, pp. 210-215.
- [22] 葛成. 三维矢量场可视化方法研究[D].南京师范大学,2011.
- [23] Sakas G. Modeling and Animating 3D Turbulence Using Spectral Synthesis[J]. The Visual Computer, 1993,9(4),200-211.
- [24] 田玲.基于体绘制的三维资料场可视化技术研究[D].西南交通大学,2007.
- [25] N. Max, R. Crawfis and D. Williams, "Visualizing wind velocities by advecting cloud textures" Proceedings Visualization '92, 1992, pp. 179-184.
- [26] F. -G. Peng, Y. Liu, D. -H. Ji, J. -R. Liu and G. -T. Qi, "The method of mass LIDAR point cloud visualization based on “Point Cloud Pyramid”," Proceedings of 2012 International Conference on Measurement, Information and Control, 2012, pp. 177-180.
- [27] 田永慧. 基于 LIC 的矢量场可视化方法研究及其在 NPR 中的应用[D].云南大学,2010.
- [28] REEVES W T. Particle systems-a technique for modeling a class of fuzzy objects [J]. In ACM Transactions on Graphics, 1983, 2(4): 91-108.
- [29] 李骞,范茵,刘捷.基于粒子追踪的风场可视化方法研究[J].现代测绘,2005(S1):43-45.
- [30] Reeves W T , BLAU R. Approximate and probabilistic algorithms for shading and rendering structured particle systems [J]. Computer Graphics, 1985, 19(7): 313-322.
- [31] J. J. van Wijk, "Rendering surface-particles," Proceedings Visualization '92, 1992, pp. 54-61.
- [32] J. Chandler, H. Obermaier and K. I. Joy, "Interpolation-Based Pathline Tracing in Particle-Based Flow Visualization" in IEEE Transactions on Visualization and Computer Graphics, vol. 21, no. 1, pp. 68-80, 1 Jan. 2015, doi: 10.1109/TVCG.2014.2325043.
- [33] Sim K. Particle Animation and Rendering Using Data Parallel Computation[J]. Computer Graphics, 1990, 24(5):5-12.
- [34] 张芹,吴慧中,张健.基于粒子系统的建模方法研究[J].计算机科学,2003(08):144-146.

- [35] 刘恒星. 基于粒子系统的 Web 数字地球海洋矢量场数据动态可视化方法[D].中国科学院大学(中国科学院遥感与数字地球研究所),2017.
- [36] S. Grottel, M. Krone, C. Müller, G. Reina and T. Ertl, "MegaMol—A Prototyping Framework for Particle-Based Visualization" in IEEE Transactions on Visualization and Computer Graphics, vol. 21, no. 2, pp. 201-214, 1 Feb. 2015.
- [37] 方京,艾波,辛文鹏,尚恒帅.粒子系统的流场动态制图优化方法[J].测绘科学,2018,43(12):72-76+84.
- [38] 韩吉烨. 基于 SPH 方法的虚拟水流墙系统中水流的模拟研究[D].东北林业大学,2015.
- [39] Van Wijk JJ. Spotnoise-Textures Synthesis for Data Visualization[J]. T W Sederberg(ed), Computer Graphics(Proceedings of ACM SIGGRAPH91), ACM, 1991(25): 309-318,1991.
- [40] O. Khorloo, Z. Gunjee, B. Sosorbaram, and N. Chiba, "Wind Field Synthesis for Animating Wind-induced Vibration", IJVR, vol. 10, no. 1, pp. 53–60, Jan. 2011.
- [41] CabralB,LeedomLC. Imaging vectorfield using line integral convolution[J].Computer Graphics(SIGGRAPH'93 Proceedings), 1993(27): 263-272.
- [42] S. Dwivedi, Mansi, L. Malhotra and R. Dubey, "3D visualization of objects using texture based algorithm," 2016 International Conference on Recent Advances and Innovations in Engineering (ICRAIE), 2016, pp. 1-4.
- [43] 王海洋. LIC 纹理可视化技术在流场中的研究与应用[D]. 西南科技大学, 2016.
- [44] Y. Liao, H. Matsui, O. Kreylos and L. H. Kellogg, "A Flow Visualization Using Parallel 3D Line Integral Convolution for Large Scale Unstructured Grid Data," 2018 IEEE 8th Symposium on Large Data Analysis and Visualization (LDAV), 2018, pp. 106-107.
- [45] Wen Zhang and Xiaomei Li, "Enhanced streamline texture synthesis for vector field visualization" Proceedings Fourth International Conference/Exhibition on High Performance Computing in the Asia-Pacific Region, 2000, pp. 782-785 vol.2.
- [46] J. Huang, Z. Pan, G. Chen, W. Chen and H. Bao, "Image-Space Texture-Based Output-Coherent Surface Flow Visualization," in IEEE Transactions on Visualization and Computer Graphics, vol. 19, no. 9, pp. 1476-1487, Sept. 2013.
- [47] M.Maskey and T. S. Newman, "Directional Texture for Visualization - New Technique and Application Study" 2015 19th International Conference on Information Visualisation, 2015, pp. 1-8.
- [48] Sunghee Kim, H. Hagh-Shenas and V. Interrante, "Conveying shape with texture: an experimental investigation of the impact of texture type on shape categorization judgments" IEEE Symposium on

- Information Visualization 2003 (IEEE Cat. No.03TH8714), 2003, pp. 163-170.
- [49] H. H. Wang, H. -X. Xu, L. Zeng and S. -K. Li, "Fuzzy Feature Visualization of Vector Field by Entropy-Based Texture Adaptation" 2011 International Conference on Virtual Reality and Visualization, 2011, pp. 303-306.
- [50] J. W. V. Dambros, W. C. Marques and C. E. Stringari, "Evaluation of the Numerical Method Runge-Kutta Used in the Oil Dispersion ECOS Model," 2014 Symposium on Automation and Computation for Naval, Offshore and Subsea (NAVCOMP), 2014, pp. 38-41.
- [51] 卢祥,袁运彬,蒋振伟.基于龙格库塔法的 GLONASS 轨道仿真研究[J].大地测量与地球动力学,2014,34(03):137-141.
- [52] Huo Qian, Li Zhong, Xi Bo and Guan Zhenzhen, "The research on calculation of water surface profile in channel by Runge - Kutta method" 2011 International Symposium on Water Resource and Environmental Protection, 2011, pp. 408-412.
- [53] H. Liu and K. Zhang, "Multi-symplectic Runge-Kutta-type methods for Hamiltonian wave equations" in IMA Journal of Numerical Analysis, vol. 26, no. 2, pp. 252-271, April 2006.
- [54] M. F. Fathoni and A. I. Wuryandari, "Comparison between Euler, Heun, Runge-Kutta and Adams-Bashforth-Moulton integration methods in the particle dynamic simulation" 2015 4th International Conference on Interactive Digital Media (ICIDM), 2015, pp. 1-7.
- [55] C. Yang and L. Qingyang, "A-stable explicit nonlinear Runge-Kutta methods" in Tsinghua Science and Technology, vol. 3, no. 4, pp. 1227-1232, Dec. 1998.
- [56] A. Al-Shimmery, "Numerical Solution of Initial Value Problems of Time-Fractional Order via a Novel Fractional 4-Stage Runge-Kutta Method" 2021 7th International Conference on Contemporary Information Technology and Mathematics (ICCITM), 2021, pp. 279-284.
- [57] K. Uçak, "A Runge-Kutta MLP Neural Network Based Control Method for Nonlinear MIMO Systems" 2019 6th International Conference on Electrical and Electronics Engineering (ICEEE), 2019, pp. 186-192.
- [58] 富明慧,梁华力.一种改进的精细-龙格库塔法[J].中山大学学报(自然科学版),2009,48(05):1-5.
- [59] Yi-Jen Wang and Chin-Teng Lin, "Runge-Kutta neural network for identification of dynamical systems in high accuracy" in IEEE Transactions on Neural Networks, vol.9,no.2, pp. 294-307, March 1998.
- [60] 李聪仁. 基于 Geotrellis 的遥感影像数据存储与检索模型设计与实现[D].云南师范大学,2018.DOI:10.27459/d.cnki.gynfc.2018.000018.
- [61] Analytical Graphics, Inc. (AGI). Cesium[EP/OL]. [2015-5-12]. <http://cesium.org>.

- [62] <https://earth.nullschool.net/#current/wind/surface/level/orthographic=-246.15,15.00,436>
- [63] <https://www.windy.com/?28.802,79.181,4>
- [64] <https://map.tianditu.gov.cn/>
- [65] 李海生,牛文杰,杨钦,陈其明.矢量场可视化的研究现状与发展趋势[J].计算机应用研究,2001(08):11-14.
- [66] 石教英 蔡文立.科学计算可视化算法与系统[M].科学出版社,1996.
- [67] 唐泽圣. 三维数据场可视化[M]. 清华大学出版社,1999.
- [68] 许庆功. 基于 GPU 的流场数据体绘制技术研究[D].西安建筑科技大学,2008.
- [69] 查石祥. NetCDF 数据的程序设计式访问[J].气象科技,2004,32(增刊):40-43.
- [70] <https://pro.arcgis.com/zh-cn/pro-app/latest/help/data/multidimensional/essential-netcdf-vocabulary.htm>.
- [71] <https://www.noaa.gov/>
- [72] 田永慧. 基于 LIC 的矢量场可视化方法研究及其在 NPR 中的应用[D].云南大学,2010.
- [73] 刘振东,刘海行,苏天赟,贾贞,李新放.基于 LIC 的海流动态模拟与可视化[J].海洋通报,2019,38(02):179-185.
- [74] Y. Yang, L. Zeng and H. Leung, "Keyframe Extraction from Motion Capture Data for Visualization" 2016 International Conference on Virtual Reality and Visualization (ICVRV), 2016, pp. 154-157.
- [75] 刘恒星. 基于粒子系统的 Web 数字地球海洋矢量场数据动态可视化方法[D].中国科学院大学 (中国科学院遥感与数字地球研究所),2017.
- [76] 沈震宇,范茵,陶俐君,胡磊.可视化技术在气象数据场分析中的运用[J].系统仿真学报,2006(S1):328-329+332.
- [77] 桂萌,葛畅.估算典型 4 阶系统动态特性经验公式的改进[J].电脑编程技巧与维护,2017(06):23-25.
- [78] Buning,P.G.Sources of error in the graphical analysis of CFD results.J Sci Comput 3,149-164 (1988).
- [79] 张文. 矢量场可视化算法研究与系统设计[D].国防科学技术大学,2001.
- [80] <https://www.ruanyifeng.com/blog/2018/07/web-worker.html>
- [81] <https://www.unrealengine.com/zh-CN/unreal-engine-5>
- [82] <https://learn.microsoft.com/en-us/windows/win32/direct3d>
- [83] Patrick Cozzi.Graphics-Tech-in-Cesium-Stack.[EB/OL]. [2015-05-26].
<http://cesiumjs.org/2015/05/26/Graphics-Tech-in-Cesium-Stack/>.
- [84] <http://nodejs.cn/learn/introduction-to-nodejs>.
- [85] 张硕. 基于感性工学的字体情感意象研究与应用[D].北京邮电大学,2019.

- [86] 张凯,王春光,刘涛,康飞龙.基于计算机视觉技术育肥猪体重分析研究[J].农机化研究,2017,39(05):32-36.DOI:10.13427/j.cnki.njyi.2017.05.006.
- [87] earth :: a global map of wind, weather, and ocean conditions (nullschool.net)

附录

附录 1 粒子渲染的顶点着色器算法

Appendix 1 Vertex Shader Algorithms for Particle Rendering

算法：segmentVertex()

```
uniform sampler2D previousParticlesPosition;
uniform sampler2D currentParticlesPosition;
uniform sampler2D postProcessingPosition;
uniform float particleHeight;
uniform float pixelSize;
struct adjacentPoints {
    vec4 previous;
    vec4 current;
    vec4 next;
};
//计算投影坐标
vec4 calculateProjectedCoordinate(vec3 lonLatLev) {
    lonLatLev.x = mod(lonLatLev.x + 180.0, 360.0) - 180.0;
    vec3 particlePosition = convertCoordinate(lonLatLev);
    vec4 projectedCoordinate = czm_modelViewProjection * vec4(particlePosition, 1.0);
    return projectedCoordinate;
}
void main() {
    vec2 particleIndex = st;
    vec3 previousPosition = texture2D(previousParticlesPosition, particleIndex).rgb;
    vec3 currentPosition = texture2D(currentParticlesPosition, particleIndex).rgb;
    vec3 nextPosition = texture2D(postProcessingPosition, particleIndex).rgb;
    float isAnyRandomPointUsed = texture2D(postProcessingPosition, particleIndex).a +
    texture2D(currentParticlesPosition, particleIndex).a +
    texture2D(previousParticlesPosition, particleIndex).a;
    adjacentPoints projectedCoordinates;
```

```

if (isAnyRandomPointUsed > 0.0) {
    projectedCoordinates.previous = calculateProjectedCoordinate(previousPosition);
    projectedCoordinates.current = projectedCoordinates.previous;
    projectedCoordinates.next = projectedCoordinates.previous;
} else {
    projectedCoordinates.previous=calculateProjectedCoordinate(previousPosition);
    projectedCoordinates.current=calculateProjectedCoordinate(currentPosition);
    projectedCoordinates.next = calculateProjectedCoordinate(nextPosition);
}
int pointToUse = int(normal.x);
float offsetSign = normal.y;
vec4 offset = vec4(0.0);
offset=pixelSize*calculateOffsetOnNormalDirection(projectedCoordinates.previous,
projectedCoordinates.current, offsetSign);
gl_Position = projectedCoordinates.previous + offset;
}

```

附录 2 粒子速度片元着色器算法

Appendix 2 Particle Velocity Fragment Shader Algorithms

算法：calculateParticlesSpeed()

```

// the size of UV textures: width = lon, height = lat*lev
uniform sampler2D U; // eastward wind
uniform sampler2D V; // northward wind
uniform sampler2D currentParticlesPosition; // (lon, lat, lev)
uniform vec3 dimension; // (lon, lat, lev)
uniform vec3 minimum; // 每个尺度的最小值
uniform vec3 maximum; // 每个尺度的最大值
uniform vec3 interval; // 每个尺度的间隔
// used to calculate the wind norm
uniform vec2 uSpeedRange; // (min, max);
uniform vec2 vSpeedRange;

```

```

uniform float pixelSize;
uniform float speedFactor;
float speedScaleFactor = speedFactor * pixelSize;
varying vec2 v_textureCoordinates;
vec3 convertSpeedUnitToLonLat(vec3 lonLatLev, vec3 speed) {
    vec2 lonLatLength = lengthOfLonLat(lonLatLev);
    float u = speed.x / lonLatLength.x;
    float v = speed.y / lonLatLength.y;
    float w = 0.0;
    vec3 windVectorInLonLatLev = vec3(u, v, w);
    return windVectorInLonLatLev;
}
void main() {
    vec3 lonLatLev = texture2D(currentParticlesPosition, v_textureCoordinates).rgb;
    vec3 speed = calculateSpeedByRungeKutta4(lonLatLev);
    vec3 speedInLonLat = convertSpeedUnitToLonLat(lonLatLev, speed);
    vec4 particleSpeed = vec4(speedInLonLat, calculateWindNorm(speed/ speedScaleFactor));
    gl_FragColor = particleSpeed;
}

```

附录 3 粒子轨迹片元着色器算法

Appendix 3 Particle Trajectory Fragment Shader Algorithms

算法：postProcessingPosition()

//粒子坐标

```

uniform sampler2D nextParticlesPosition;
uniform sampler2D particlesSpeed; // (u, v, w, norm)
// range (min, max)
uniform vec2 lonRange;
uniform vec2 latRange;
to avoid degeneration
uniform float dropRateBump;

```

```
varying vec2 v_textureCoordinates;

const vec3 randomConstants = vec3(12.9898, 78.233, 4375.85453);
const vec2 normalRange = vec2(0.0, 1.0);
//产生随机粒子
vec3 generateRandomParticle(vec2 seed, float lev) {
    //确保数据变化的范围在(0, 360)
    float randomLon = mod(rand(seed, lonRange), 360.0);
    float randomLat = rand(-seed, latRange);
    return vec3(randomLon, randomLat, lev);
}

void main() {
    vec3 nextParticle = texture2D(nextParticlesPosition, v_textureCoordinates).rgb;
    vec4 nextSpeed = texture2D(particlesSpeed, v_textureCoordinates);
    float speedNorm = nextSpeed.a;
    float particleDropRate = dropRate + dropRateBump * speedNorm;
    vec2 seed1 = nextParticle.xy + v_textureCoordinates;
    vec2 seed2 = nextSpeed.xy + v_textureCoordinates;
    vec3 randomParticle = generateRandomParticle(seed1, nextParticle.z);
    float randomNumber = rand(seed2, normalRange);
    if (randomNumber < particleDropRate || particleOutbound(nextParticle)) {
        gl_FragColor = vec4(randomParticle, 1.0);
    } else {
        gl_FragColor = vec4(nextParticle, 0.0);
    }
}
```
